

Name	Hamza Khan
Reg No	FA19-BCE-075
Section	BCE-2B
Labs	11 & 12
Submitted to	Mam Asma Ramay

Control Input:

Control Inputs:-

We need 4 inputs

- 1) "A" for up/down counting
- 2) "clock" for the flipflops
- 3) "Enable" for pausing/starting
- 4) "reset" as "rst" for resetting the flipflops

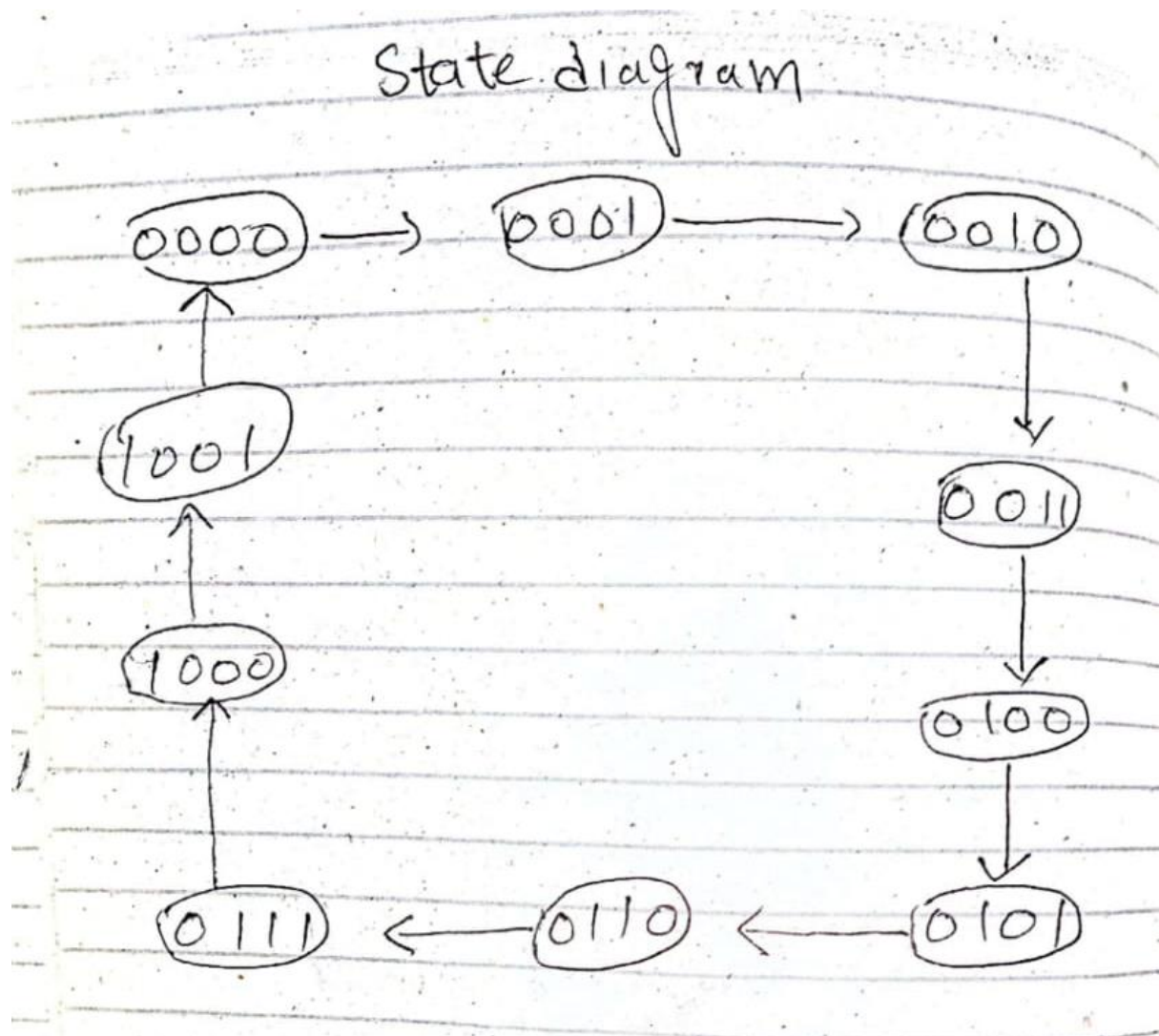
Outputs = 2

1) "out" output as bits

2) "seg" output as seven segment display codes.

Control inputs For BCD up/down Counter
--

Pre-Lab:



State diagram For BCD up Counter only

b)

Present state				Next state			
A	B	C	D	A _{t+1}	B _{t+1}	C _{t+1}	D _{t+1}
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

State table For BCD up Counter only

T flipflops

A_{t+1}	B_{t+1}	C_{t+1}	D_{t+1}
0	0	0	1
0	0	1	1
0	0	0	1
0	1	1	1
0	0	0	1
0	0	1	1
0	0	0	1
1	1	1	1
0	0	0	1
1	0	0	1
x	x	x	x
x	x	x	x
x	x	x	x
x	x	x	x
x	x	x	x
x	x	x	x
x	x	x	x

$$D_{t+1} = 1$$

AB \ CD	00	01	11	10
00				
01			1	
11	x	x	x	x
10		1	x	x

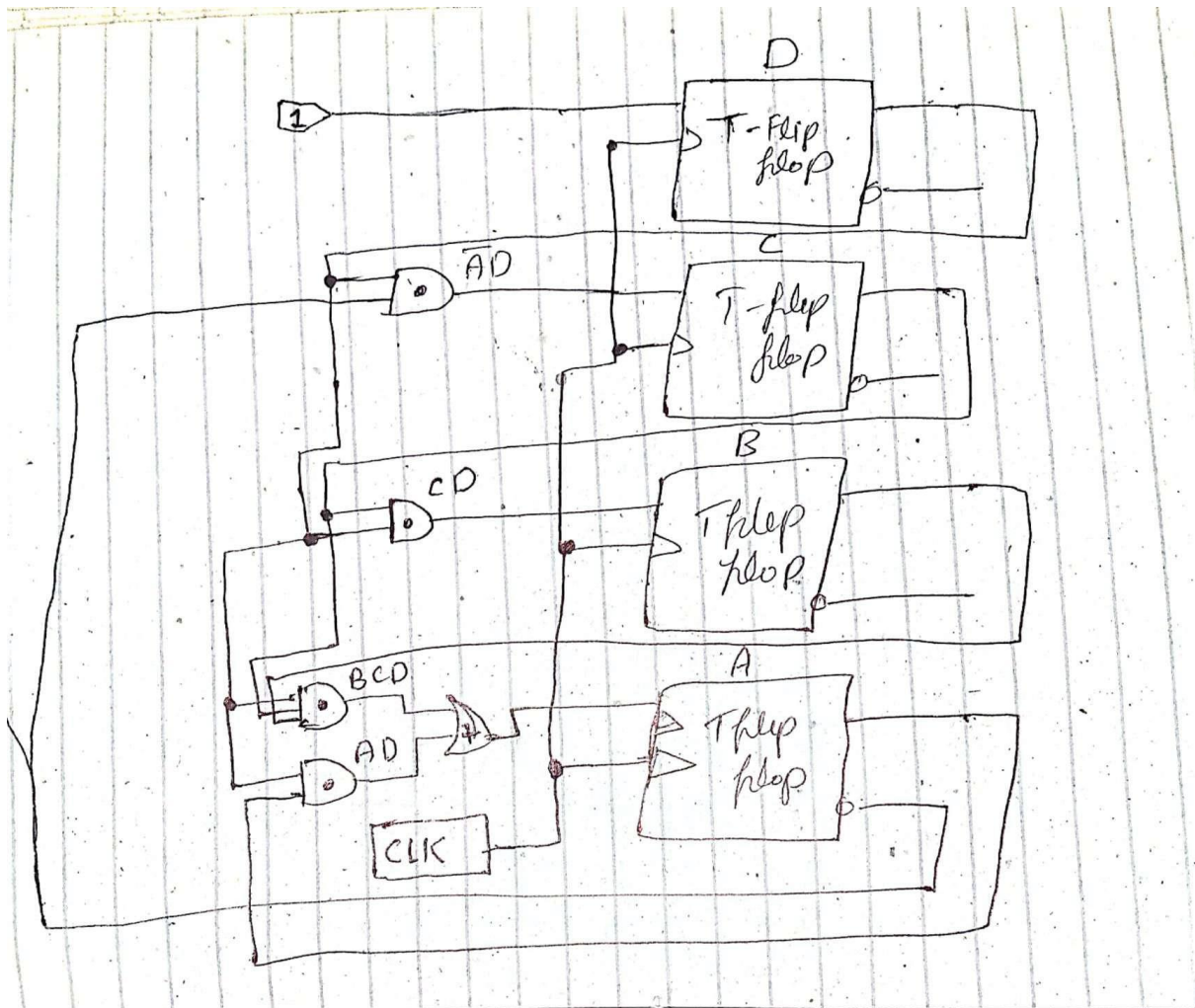
$$A_{t+1} = BCD + AD$$

AB \ CD	00	01	11	10
00			1	
01			1	
11	x	x	x	x
10			x	x

$$B_{t+1} = CD$$

AB \ CD	00	01	11	10
00		1	1	
01		1	1	
11	x	x	x	x
10			x	x

$$C_{t+1} = \overline{A}D$$



CS Scanned with CamScanner

Tflipflop circuit diagram For BCD up Counter only

(1)

AB \ CD	00	01	11	10
00				
01			1	
11	x	x	x	x
10			x	x

$$A_{tri} = BCD + A\bar{D}$$

AB \ CD	00	01	11	10
00			1	
01	1	1		1
11	x	x	x	x
10			x	x

$$B_{tri} = \overline{BCD} + \overline{BC} + \overline{BD}$$

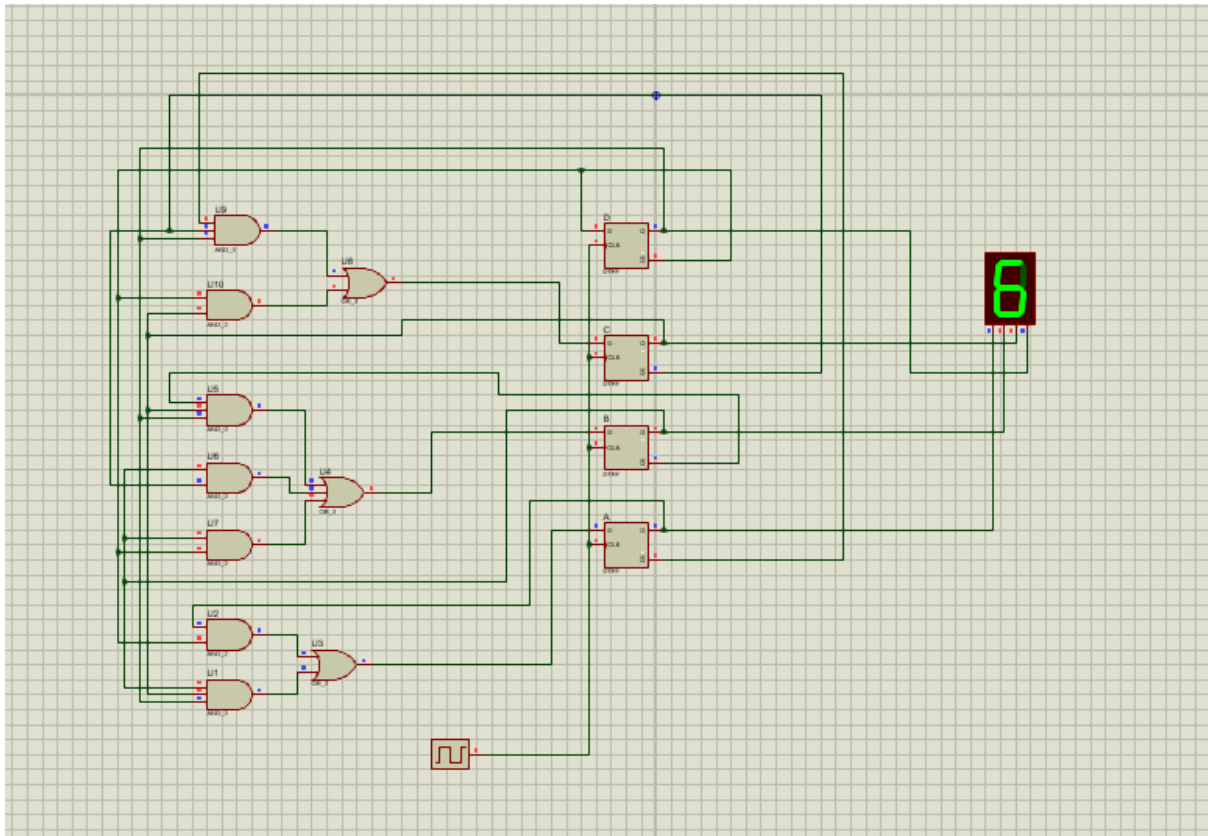
AB \ CD	00	01	11	10
00		1		1
01		1		1
11	x	x	x	x
10			x	x

$$C_{tri} = \overline{A}\overline{C}\overline{D} + C\overline{D}$$

AB \ CD				
	00	01	11	10
00	1			1
01	1			1
11	x	x	x	x
10	1		x	x

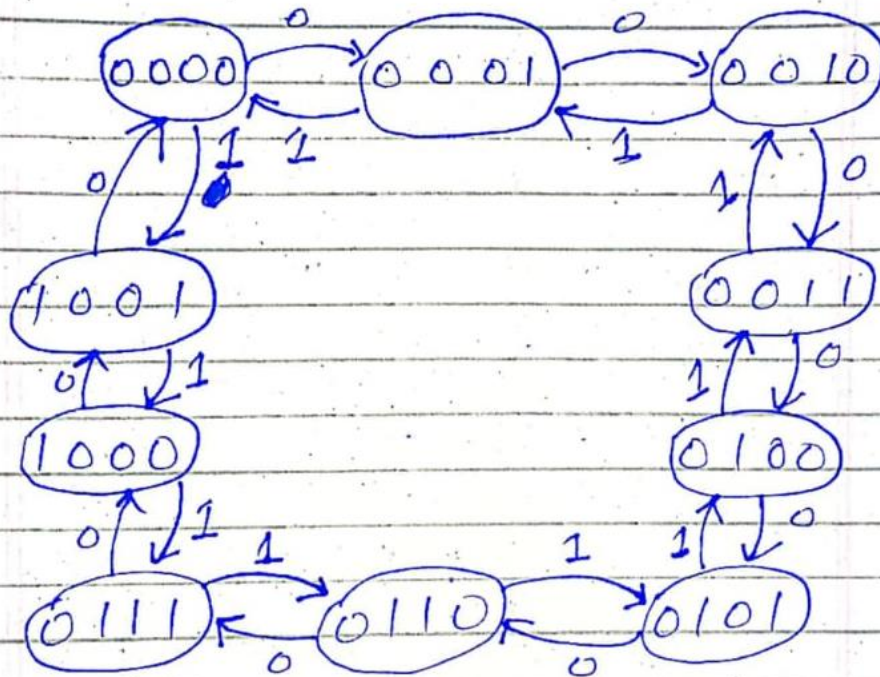
$$D_{t+1} = \overline{D}$$

Dflipflop equation For BCD up Counter only



Dflipflop circuit For BCD up Counter only

In-Lab task 1:

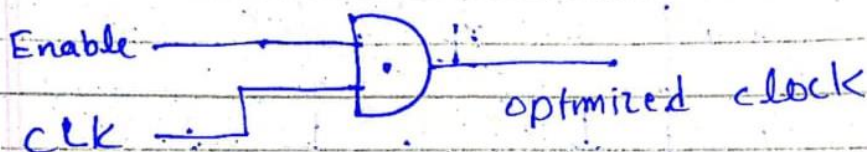


state Diagram

State Diagram For BCD up/down Counter

Also

Enable = 0 CLK = 1/0 -①
 Enable = 1 CLK = 0/1 -②



The optimized clock is used.

Also

Reset = 1 state become
 "0000", reset is applied
 to all the flipflops

Input A	Present state				Next state			
	B	C	D	E	B _{next}	C _{next}	D _{next}	E _{next}
0	0	0	0	0	0	0	0	1
0	0	0	0	1	0	0	1	0
0	0	0	1	0	0	0	1	1
0	0	0	1	1	0	1	0	0
0	0	1	0	0	0	1	0	1
0	0	1	0	1	0	1	1	0
0	0	1	1	0	0	1	1	1
0	0	1	1	1	1	0	0	0
0	1	0	0	0	1	0	0	1
0	1	0	0	1	0	0	0	0
0	1	0	1	0	x	x	x	x
0	1	0	1	1	x	x	x	x
0	1	1	0	0	x	x	x	x
0	1	1	0	1	x	x	x	x
0	1	1	1	0	x	x	x	x
0	1	1	1	1	x	x	x	x
1	0	0	0	0	1	0	0	1
1	0	0	0	1	0	0	0	0
1	0	0	1	0	0	0	0	1
1	0	0	1	1	0	0	1	0
1	0	1	0	0	0	0	1	1
1	0	1	0	1	0	1	0	0
1	0	1	1	0	0	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	0	0	0	1	1	1
1	1	0	0	1	1	0	0	0
1	1	0	1	0	x	x	x	x
1	1	0	1	1	x	x	x	x
1	1	1	0	0	x	x	x	x
1	1	1	0	1	x	x	x	x
1	1	1	1	0	x	x	x	x
1	1	1	1	1	x	x	x	x

State table For BCD up/down Counter

ABCE

	00	01	11	10
000				
001			1	
011	x	x	x	x
010	1		x	x
110	1			
111				
101	x	x	x	x
100		1	x	x

$$B_{t+1} = \overline{A}\overline{B}\overline{E} + A\overline{B}\overline{E} + \overline{A}CDE + A\overline{C}\overline{D}\overline{E}\overline{B}$$

	00	01	11	10
000			1	
001	1	1		1
011	x	x	x	x
010			x	x
110				
111		1	1	1
101	x	x	x	x
100	1		x	x

$$C_{t+1} = \overline{A}\overline{C}\overline{D} + CDE + ACE + A\overline{B}\overline{E} + \overline{A}\overline{C}DE$$

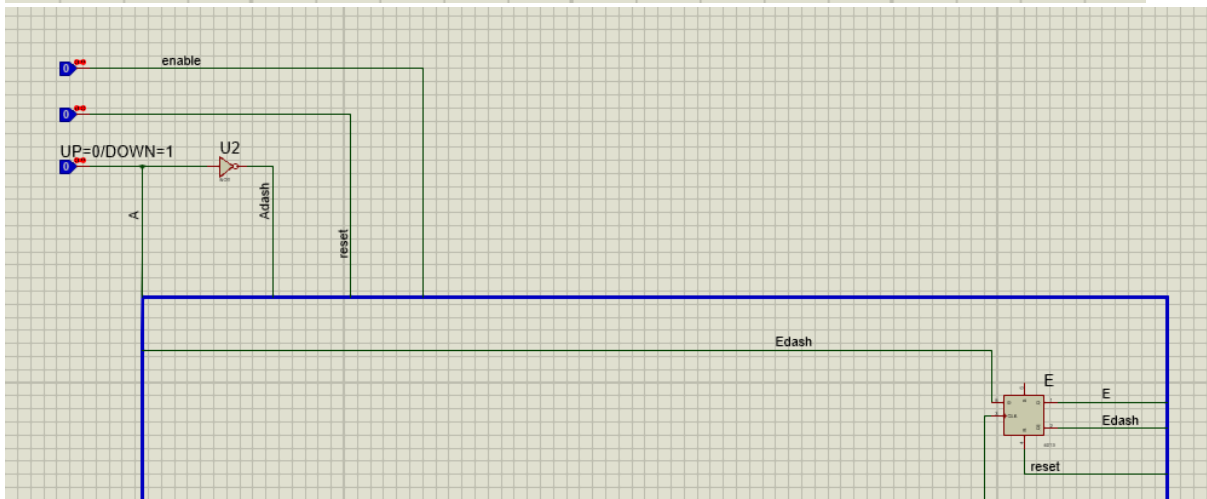
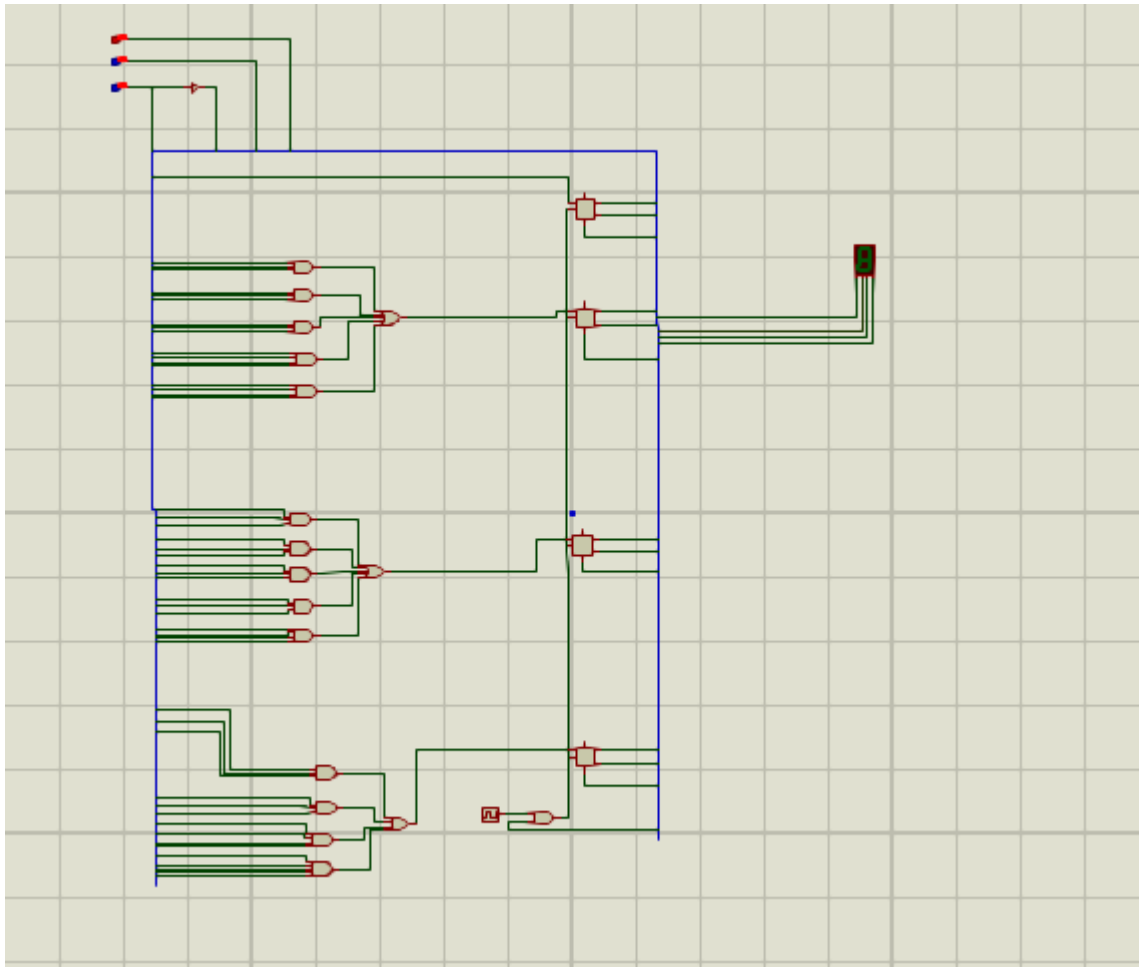
ABC \ DE	00	01	11	10
000		1		1
001		1		1
011	x	x	x	x
010			x	x
110			1	
111	1		1	
101	x	x	x	x
100	1		x	x

$$D_{t+1} = ADE + A\bar{D}\bar{E} + A\bar{B}E + \bar{A}\bar{B}\bar{D}E + AC\bar{D}E$$

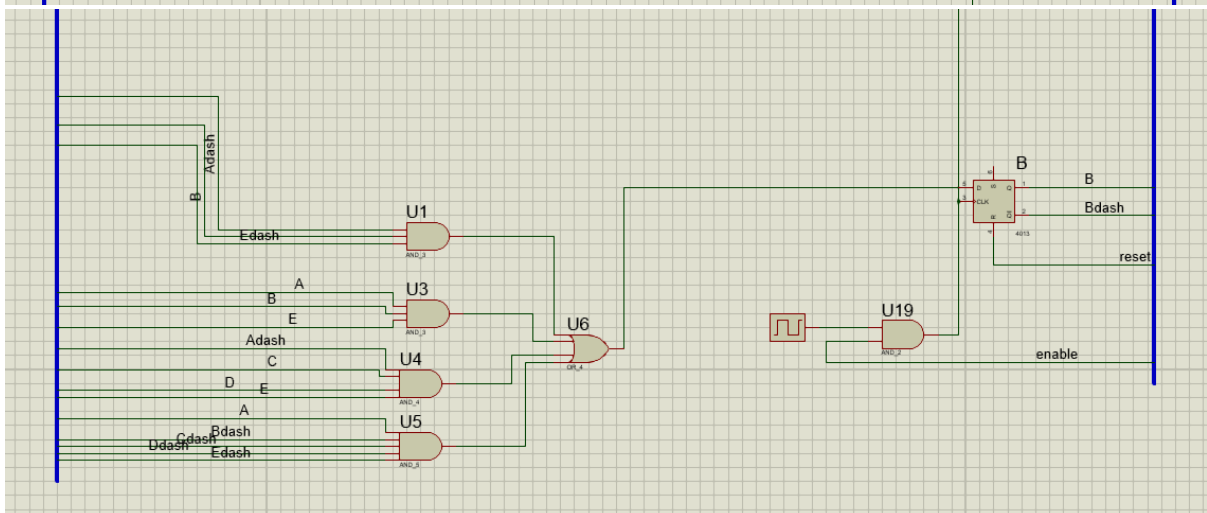
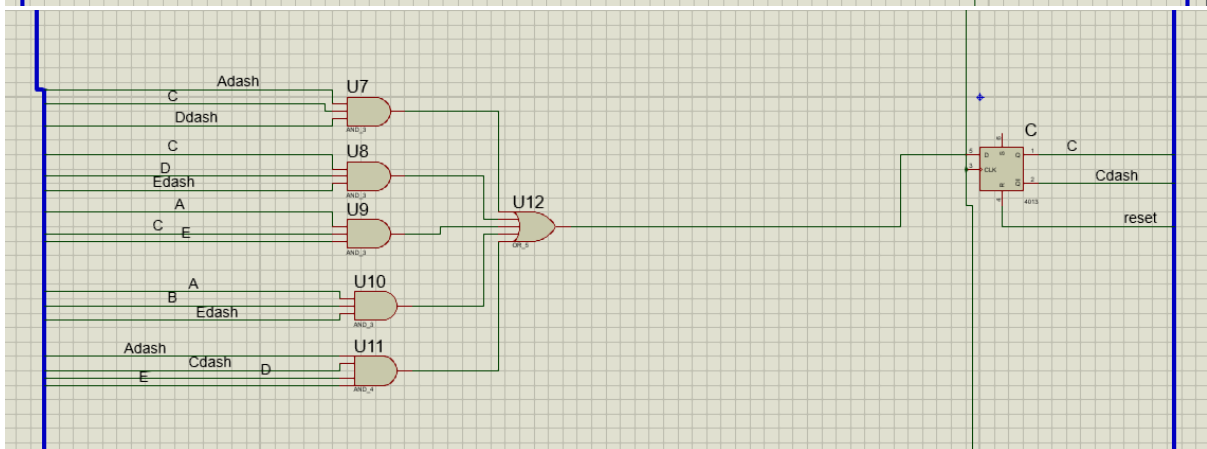
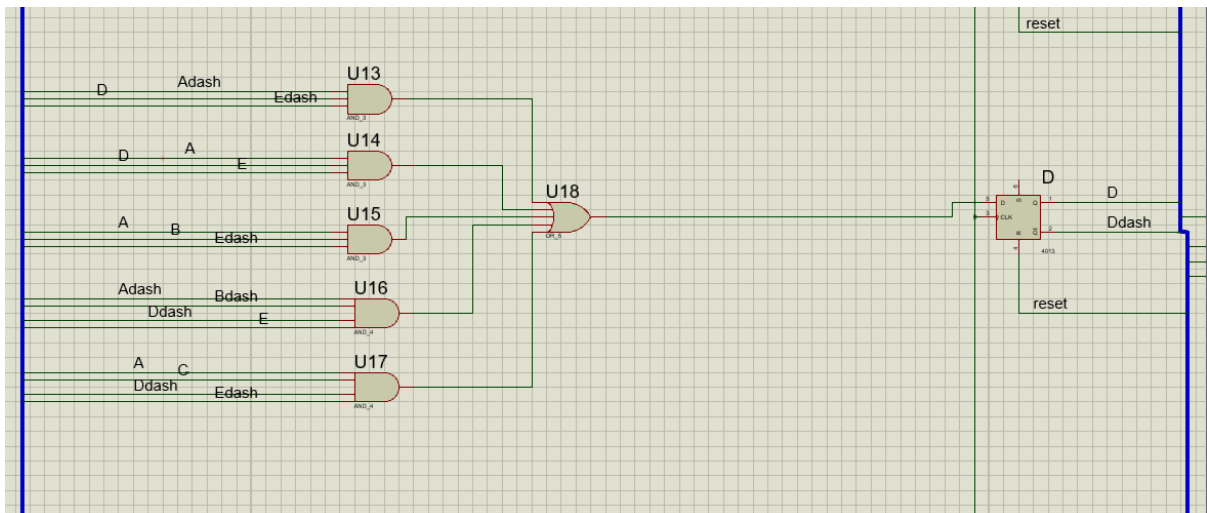
ABC \ DE	00	01	11	10
000	1			1
001	1			1
011	x	x	x	x
010	1		x	x
110	1			1
111	1			1
101	x	x	x	x
100	1		x	x

$$E_{t+1} = \bar{E}$$

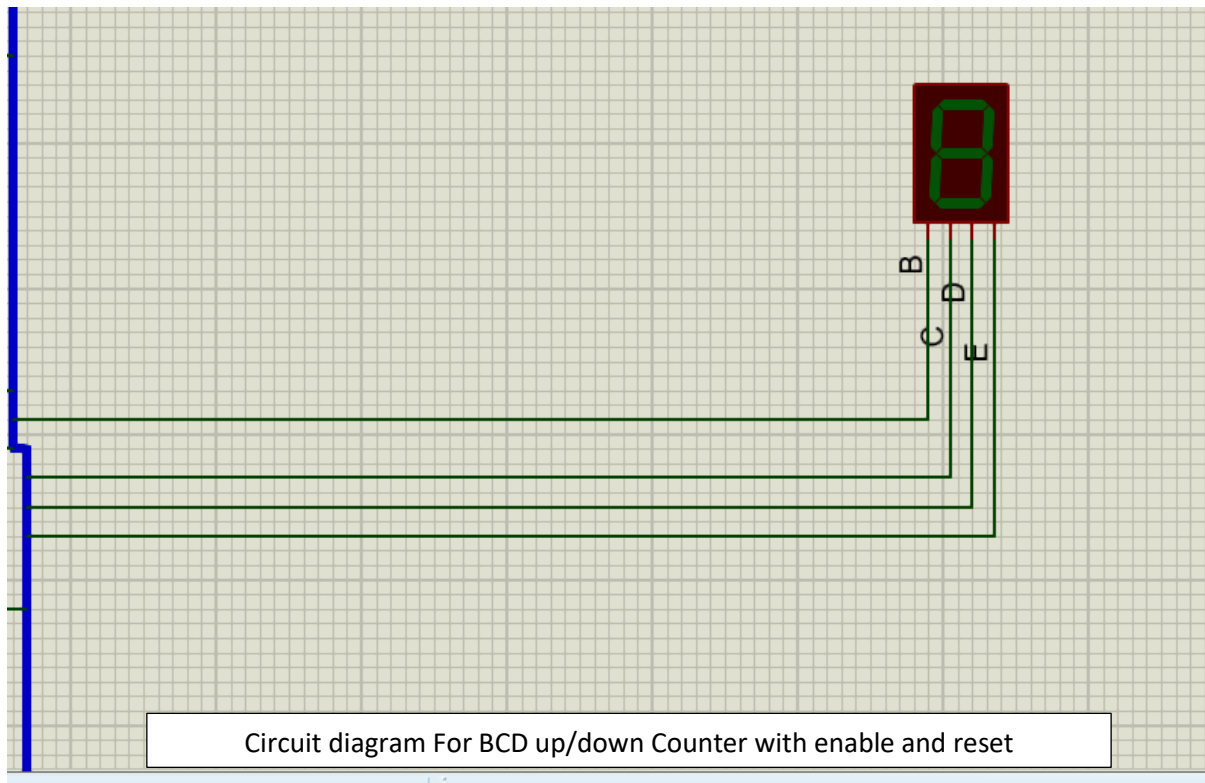
State equations For BCD up/down Counter



Circuit diagram For BCD up/down Counter with enable and reset



Circuit diagram For BCD up/down Counter with enable and reset



```

20 //////////////////////////////////////////////////
21 module dflipflop(Q,D,clk,rst
22 );
23 output reg Q;
24 input D,clk,rst;
25 always@(posedge clk or posedge rst)
26 begin
27 if(rst)begin Q<=1'b0; end
28 else begin Q<=D; end
29 end
30 endmodule
31

```

Dflipflop code (that is to be used in these labs)

```

21 module bcdto7segment(
22     bcd,segments );
23
24 input [3:0] bcd;
25 output reg [7:0]segments;
26 always@(bcd)begin
27 case(bcd)
28 0: segments=8'b1000000; //Dp is MSb and A is LSB
29 1: segments=8'b11111001;
30 2: segments=8'b10100100;
31 3: segments=8'b10110000;
32 4: segments=8'b10011001;
33 5: segments=8'b10010010;
34 6: segments=8'b10000010;
35 7: segments=8'b11111000;
36 8: segments=8'b10000000;
37 9: segments=8'b10010000;
38 default: segments=8'b11111111;
39 endcase
40 end
41 endmodule
42

```

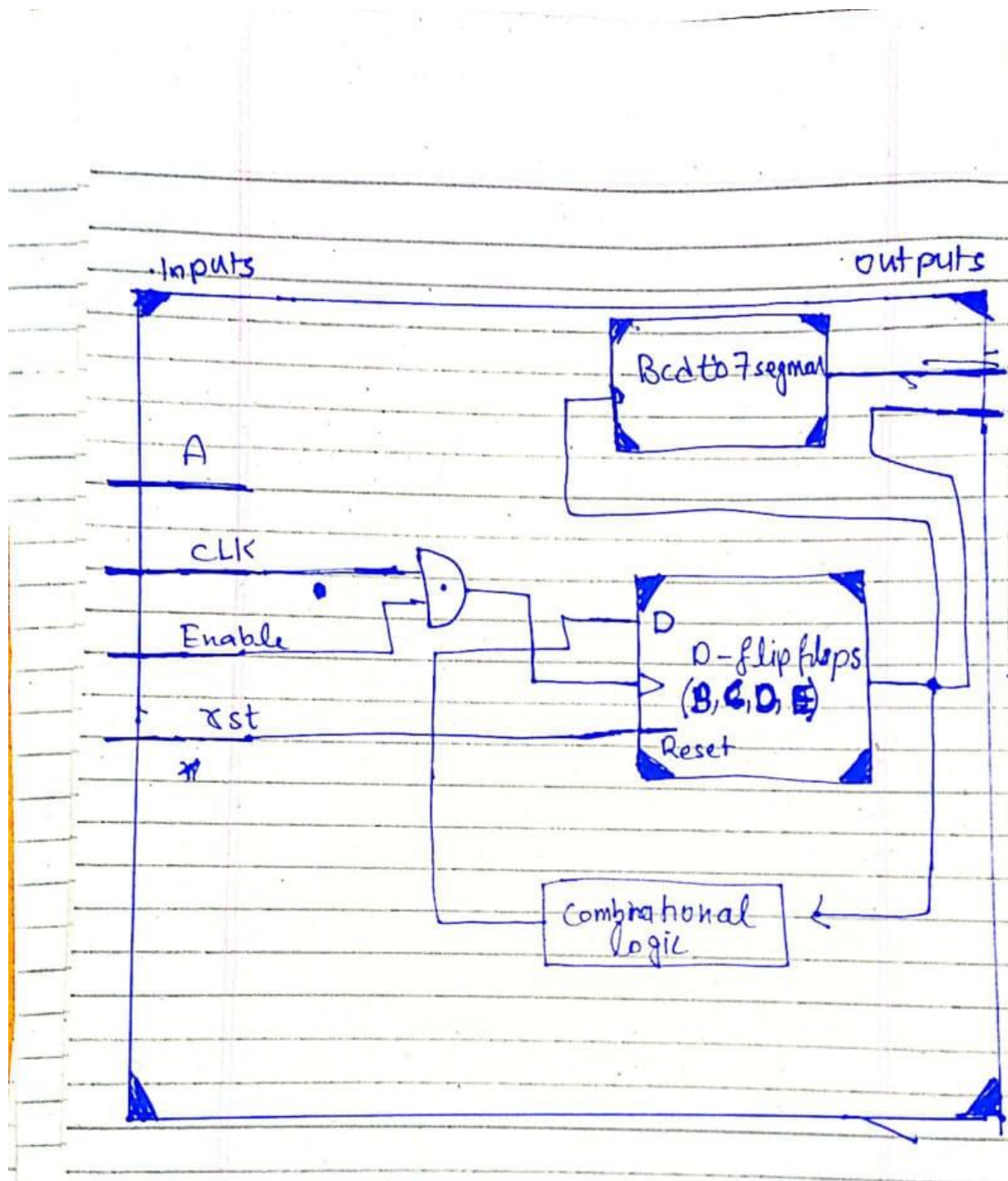
7 Segment display code (that is to be used in these labs)

```

20 //////////////////////////////////////////////////
21 module updowncounter(A,clk,enable,out,rst,seg);
22 assign anode=4'b0001;//for SEVENsegmentDisp on FPGA
23 output [3:0]out; //output
24 input A,clk,rst,enable;//input,reset,enable and clock
25 output [7:0]seg;//seven segment output
26 wire [4:0]dash;//for the inverse of state and input
27 wire [3:0]B; //temporary between the outputs of fliflop and inputs to gates and inturn the fliflops
28 wire secClk;
29
30 and clk1(secClk,clk,enable);//clock optimized for enable/disable
31
32 not n1(dash[4],A);//Adash
33 not n2(dash[3],B[3]);//Bdash
34 not n3(dash[2],B[2]);//Cdash
35 not n4(dash[1],B[1]);//Ddash
36 not n5(dash[0],B[0]);//Edash
37
38 wire [5:0]Dr;//for Dfliflop
39 and D1(Dr[0],dash[4],B[1],dash[0]);
40 and D2(Dr[1],A,B[1],B[0]);
41 and D3(Dr[2],A,B[3],dash[0]);
42 and D4(Dr[3],dash[4],dash[3],dash[1],B[0]);
43 and D5(Dr[4],A,B[2],dash[1],dash[0]);
44 or Dflip(Dr[5],Dr[0],Dr[1],Dr[2],Dr[3],Dr[4]);
45
46 wire [5:0]Cr;//for Cflilop
47 and C1(Cr[0],dash[4],B[2],dash[1]);
48 and C2(Cr[1],B[2],B[1],dash[0]);
49 and C3(Cr[2],A,B[2],B[0]);
50 and C4(Cr[3],A,B[3],dash[0]);
51 and C5(Cr[4],dash[4],dash[2],B[1],B[0]);
52 or Cflip(Cr[5],Cr[0],Cr[1],Cr[2],Cr[3],Cr[4]);
53
54 wire [4:0] Br; //for Bfliflop
55 and B1(Br[0],dash[4],B[3],dash[0]);
56 and B2(Br[1],A,B[3],B[0]);
57 and B3(Br[2],dash[4],B[2],B[1],B[0]);
58 and B4(Br[3],A,dash[3],dash[2],dash[1],dash[0]);
59 or Bflip(Br[4],Br[3],Br[2],Br[1],Br[0]);
60
61 //E,D,C,B fliflops
62 dflipflop Eflop(B[0],dash[0],secClk,rst);
63 dflipflop Dflop(B[1],Dr[5],secClk,rst);
64 dflipflop Cflop(B[2],Cr[5],secClk,rst);
65 dflipflop Bflop(B[3],Br[4],secClk,rst);
66
67 assign out[3:0]={B[3],B[2],B[1],B[0]};//ouput assigned
68 bcdto7segment bcd1(out,seg);//sevensegment output calculated and assigned
69
70 endmodule

```

Up/down counter Code with enable and reset



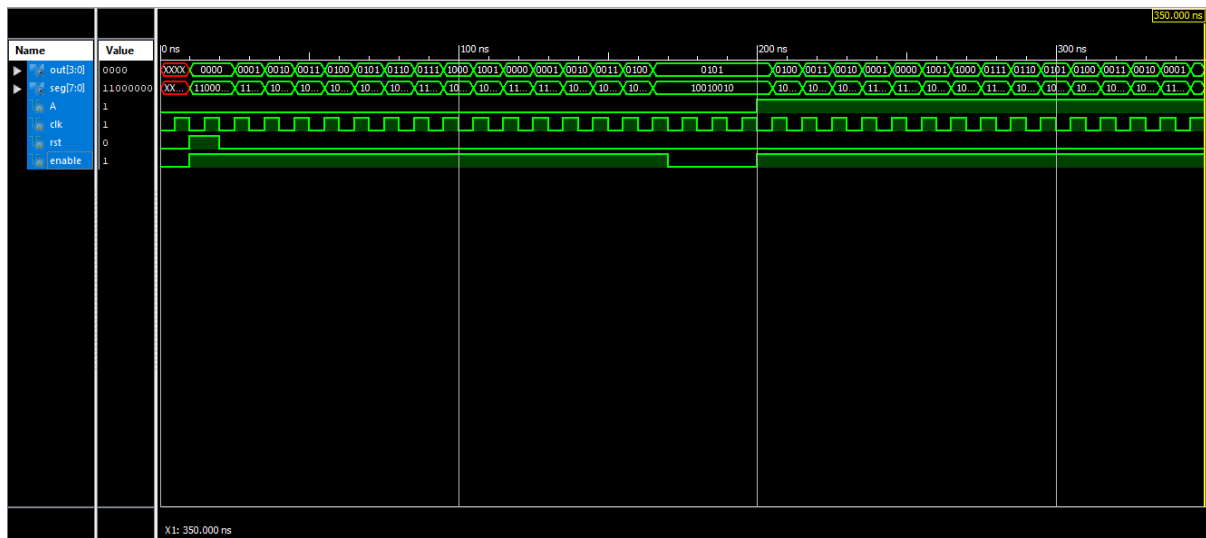
Up/down counter with enable/reset Block diagram

```

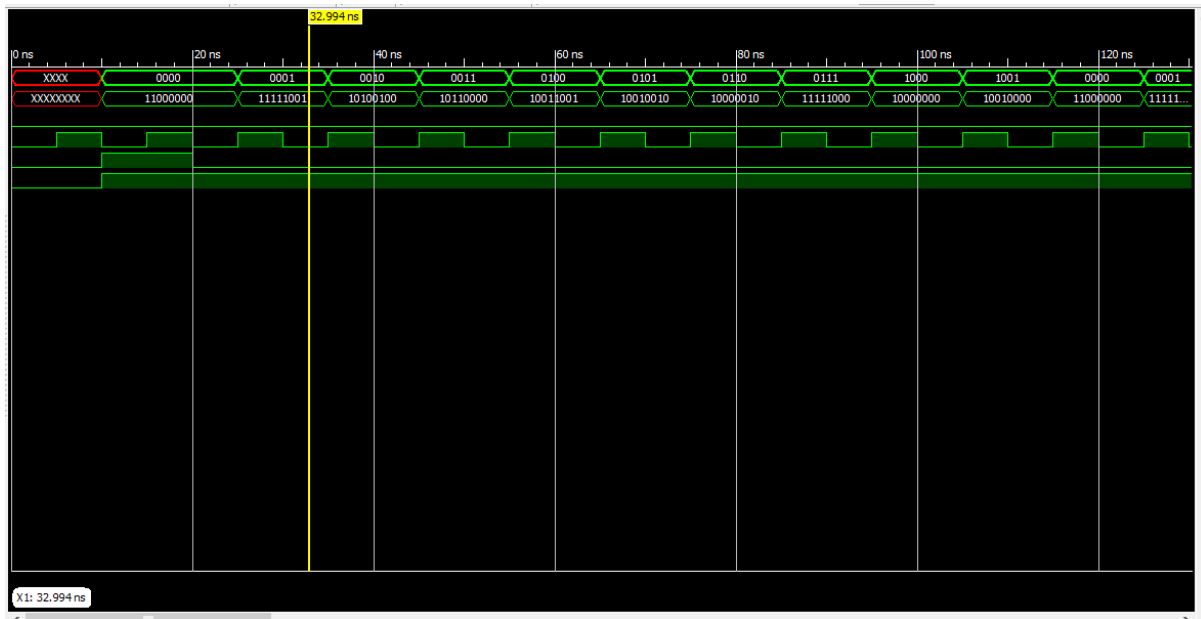
25 module testbn;
26     // Inputs
27     reg A;
28     reg clk;
29     reg rst;
30     reg enable;
31     // Outputs
32     wire [3:0] out;
33     wire [7:0] seg;
34
35     // Instantiate the Unit Under Test (UUT)
36     updowncounter uut (
37         .A(A),
38         .clk(clk),
39         .enable(enable),
40         .out(out),
41         .rst(rst),
42         .seg(seg)
43     );
44     initial begin
45         A = 0; //upcount
46         clk = 0;
47         enable=0;//disabled
48         rst=0; #10;
49         enable=1; //enabled
50         rst=1; #10;//reset
51         rst=0;
52         enable=1; #150;//enabled
53         enable=0; #30; //disabled
54         enable=1; //enabled
55         A=1; #150;//downcount
56         $finish;
57     end
58     always #5 clk=~clk;
59 endmodule

```

Up/down counter with enable/reset Testbench



Up/down counter with enable/reset Graph 1



Up/down counter with enable/reset Graph 2

In-lab task 2:

```

20 ///////////////////////////////////////////////////////////////////
21 module behavcounter(
22     A,enable,clk,reset,state,seg );
23 parameter A0=4'b0000,A1=4'b0001,A2=4'b0010,A3=4'b0011,A4=4'b0100,A5=4'b0101,
24     A6=4'b0110,A7=4'b0111,A8=4'b1000,A9=4'b1001;//the 9 states
25 assign anode=4'b0001;//for SEVENsegmentDisp on FPGA
26 output reg [3:0]state;//the currentstate output
27 output [7:0]seg;//seven segment output
28 input A,reset,clk,enable;//inout, enable,clock,reset
29
30 //positive edge clock and reset and negative edge enable
31 always@(posedge clk or posedge reset or negedge enable)
32 begin
33     if(reset==1)begin state<=A0; end//reset
34     else if(enable==0)begin state<=state; end//enable/disable
35     else
36     begin
37         case (state)
38         //states with coresponding next states according to inpuy
39         //A zero =up count / A one=down count
40         A0:begin if(A==0)begin state<=A1; end else begin state<=A9; end end
41         A1:begin if(A==0)begin state<=A2; end else begin state<=A0; end end
42         A2:begin if(A==0)begin state<=A3; end else begin state<=A1; end end
43         A3:begin if(A==0)begin state<=A4; end else begin state<=A2; end end
44         A4:begin if(A==0)begin state<=A5; end else begin state<=A3; end end
45         A5:begin if(A==0)begin state<=A6; end else begin state<=A4; end end
46         A6:begin if(A==0)begin state<=A7; end else begin state<=A5; end end
47         A7:begin if(A==0)begin state<=A8; end else begin state<=A6; end end
48         A8:begin if(A==0)begin state<=A9; end else begin state<=A7; end end
49         A9:begin if(A==0)begin state<=A0; end else begin state<=A8; end end
50         default: state<=4'bxxxx;//default state
51         endcase
52     end
53 end
54 bcdto7segment BCl(state,seg);// for seven segment output

```

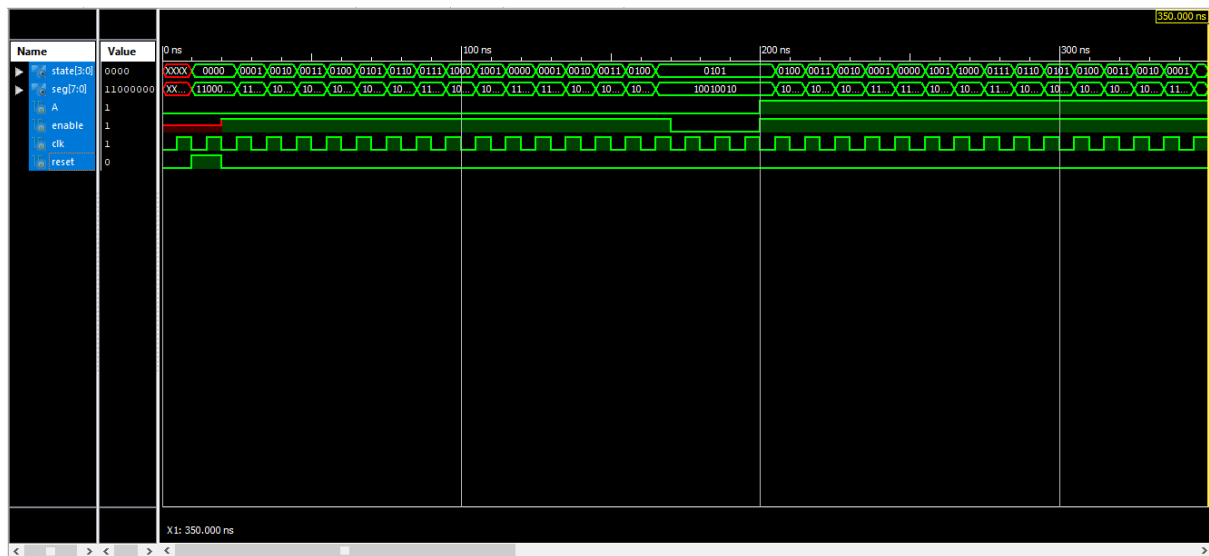
Up/down counter with enable/reset State based Behavioral code

```

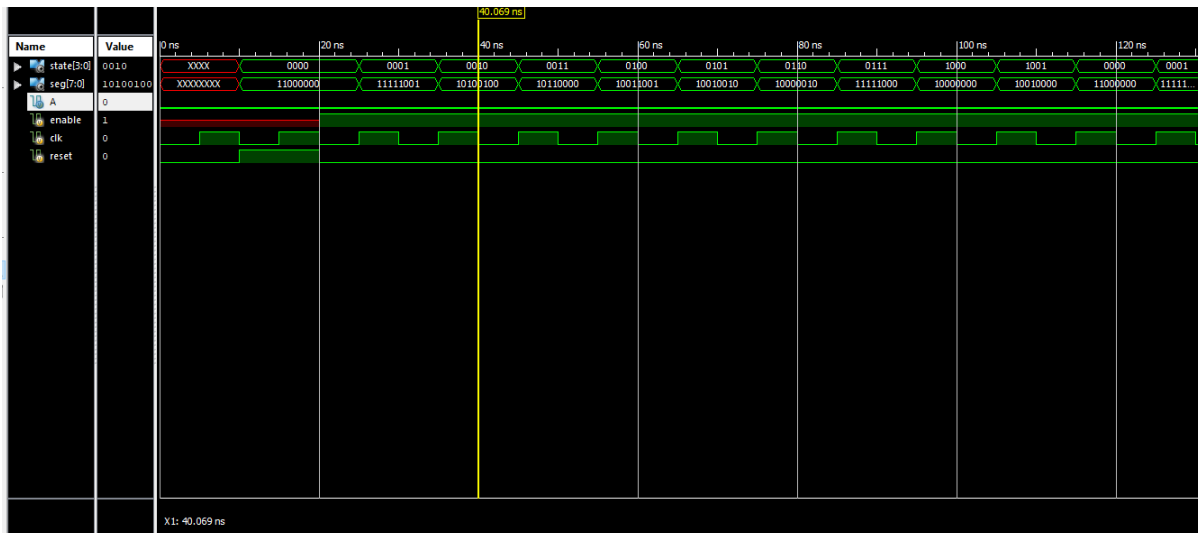
25 module testbenchbehavcount;|
26     // Inputs
27     reg A;
28     reg enable;
29     reg clk;
30     reg reset;
31
32     // Outputs
33     wire [3:0] state;
34     wire [7:0] seg;
35     // Instantiate the Unit Under Test (UUT)
36     behavcounter uut (
37         .A(A),
38         .enable(enable),
39         .clk(clk),
40         .reset(reset),
41         .state(state),
42         .seg(seg)
43     );
44     initial begin
45         // Initialize Inputs
46         A = 0; //upcount
47         clk = 0;
48         reset=0; #10;
49         reset=1; #10;//reset
50         reset=0;
51         enable=1; #150;//enabled
52         enable=0; #30; //disabled
53         enable=1; //enabled
54         A=1; #150;//downcount
55
56         $finish;
57     end
58     always #5 clk=~clk; //clock
59 endmodule

```

Up/down counter with enable/reset State based Behavioral code's Testbench



Up/down counter with enable/reset State based Behavioral code's Graph 1



Up/down counter with enable/reset State based Behavioral code's Graph 2

Post-lab task1:

```

20 ///////////////////////////////////////////////////////////////////
21 module behavlcounterr(
22     A,clk,enable,reset,out);
23     parameter n=4;           //no of bits
24     input A,clk,enable,reset;
25     output reg [n-1:0] out;   //output
26
27     //positive edge clk and reset
28     always@( posedge clk, posedge reset )
29     begin
30         if(reset ==1)begin out<=0; end           //reset
31         else if(enable==0)begin out<=out; end     //enable/disable
32         else
33         begin
34             if (A==0)begin out <=out+1; end       //upcount
35             else begin out<=out-1; end             //down count
36         end
37     end
38 endmodule
39

```

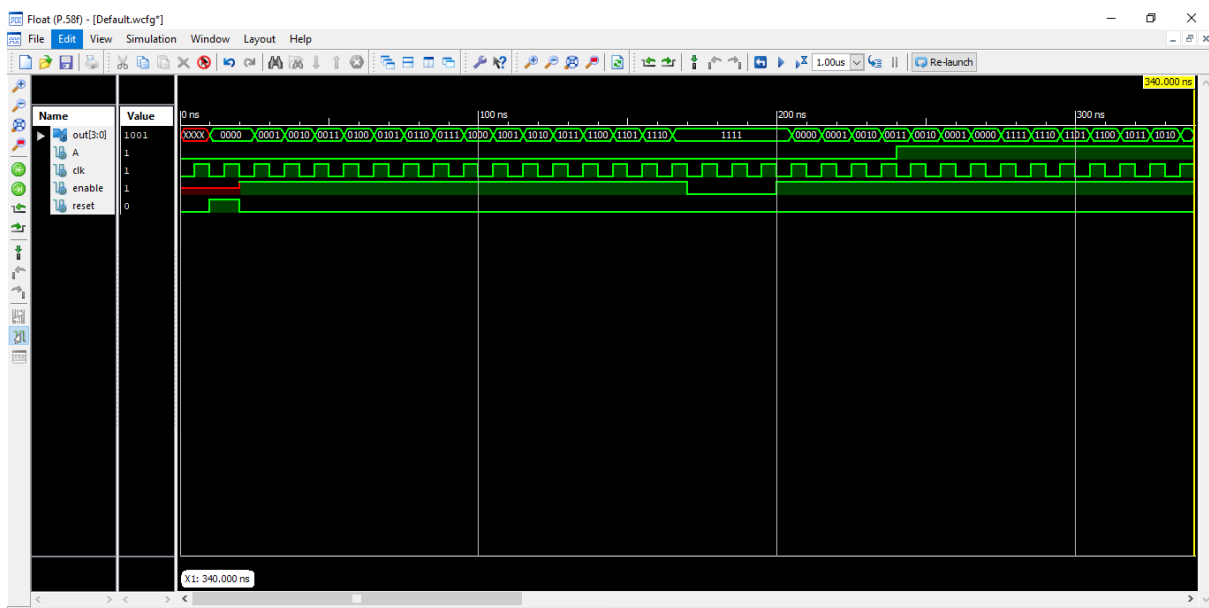
n-bit counter with enable/reset Behavioral code

```

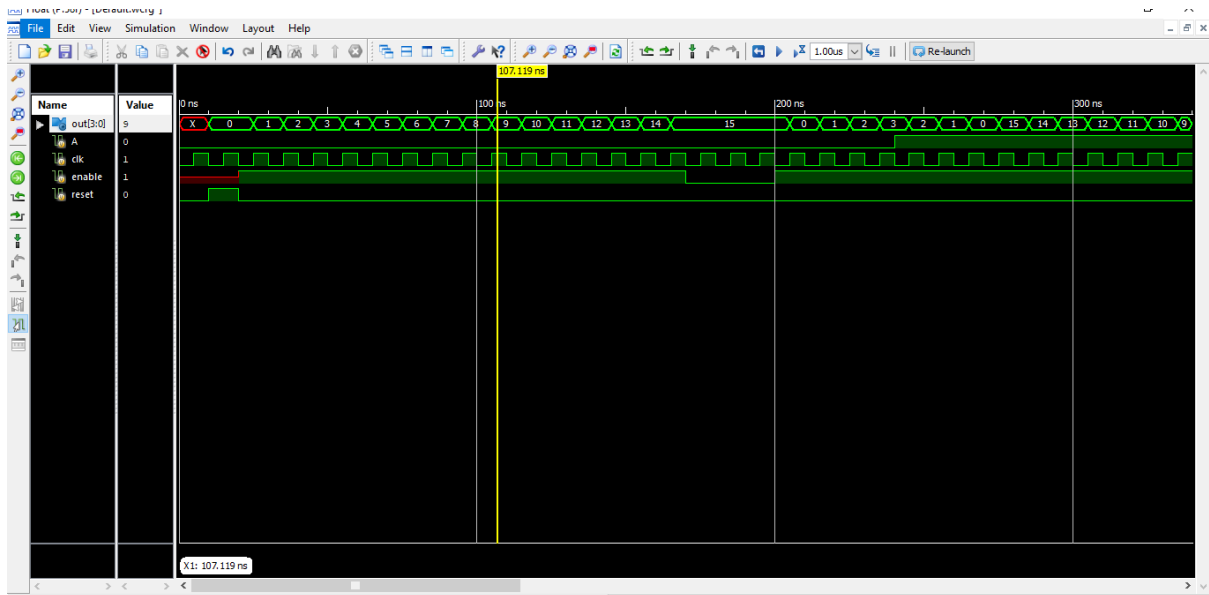
24
25 module tesbenchbehavnmodule;
26
27     // Inputs
28     reg A;
29     reg clk;
30     reg enable;
31     reg reset;
32     // Outputs
33     wire [3:0] out;
34     // Instantiate the Unit Under Test (UUT)
35     behavlcounterr uut (
36         .A(A),
37         .clk(clk),
38         .enable(enable),
39         .reset(reset),
40         .out(out)
41     );
42     initial begin
43         // Initialize Inputs
44         A = 0; //up count
45         clk = 0;
46         reset=0; #10;
47         reset=1; #10; //reset
48         reset=0;
49         enable=1; #150; //enable
50         enable=0; #30; //disable
51         enable=1; //enable
52         A=0; #40; //upcount
53         A=1; #100; //downcount
54         $finish;
55
56     end
57     always #5 clk=~clk; //clock
58 endmodule

```

n-bit counter with enable/reset Behavioral code's Testbench



n-bit counter with enable/reset Behavioral code's Graph 1



n-bit counter with enable/reset Behavioral code's Graph 1

Post-lab task 2:

Device utilization summary:

Selected Device : 3s500efg320-4

Number of Slices:	3	out of	4656	0%
Number of Slice Flip Flops:	4	out of	9312	0%
Number of 4 input LUTs:	5	out of	9312	0%
Number of IOs:	8			
Number of bonded IOBs:	8	out of	232	3%
Number of GCLKs:	1	out of	24	4%

Up/down counter with enable/reset Gate level resource summary

Device utilization summary:

Selected Device : 3s500efg320-4

Number of Slices:	7	out of	4656	0%
Number of Slice Flip Flops:	4	out of	9312	0%
Number of 4 input LUTs:	14	out of	9312	0%
Number of IOs:	16			
Number of bonded IOBs:	16	out of	232	6%
Number of GCLKs:	1	out of	24	4%

Up/down counter with enable/reset State Based behavioral code resource summary

Device utilization summary:

Selected Device : 3s500efg320-4

Number of Slices:	4	out of	4656	0%
Number of Slice Flip Flops:	4	out of	9312	0%
Number of 4 input LUTs:	8	out of	9312	0%
Number of IOs:	14			
Number of bonded IOBs:	12	out of	232	5%
Number of GCLKs:	1	out of	24	4%

n-bit counter with enable/reset behavioral code resource summary

Timing Summary:

Speed Grade: -4

Minimum period: 3.123ns (Maximum Frequency: 320.205MHz)
Minimum input arrival time before clock: 3.819ns
Maximum output required time after clock: 4.496ns
Maximum combinational path delay: No path found

Timing constraint: Default period analysis for Clock 'clk'

Clock period: 3.123ns (frequency: 320.205MHz)
Total number of paths / destination ports: 10 / 4

Delay: 3.123ns (Levels of Logic = 2)

Source: out_0 (FF)
Destination: out_3 (FF)
Source Clock: clk rising
Destination Clock: clk rising

Data Path: out_0 to out_3

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDCE:C->Q	5	0.591	0.637	out_0 (out_0)
LUT4_L:I3->LO	1	0.704	0.179	Result<3>_SW0 (N01)
LUT2:I1->O	1	0.704	0.000	Result<3> (Result<3>)
FDCE:D		0.308		out_3
Total		3.123ns (2.307ns logic, 0.816ns route) (73.9% logic, 26.1% route)		

Up/down counter with enable/reset Gate level Time summary

```
=====
Timing constraint: Default OFFSET IN BEFORE for Clock 'clk'
Total number of paths / destination ports: 7 / 7
=====
```

```
Offset:          3.819ns (Levels of Logic = 3)
Source:          A (PAD)
Destination:     out_3 (FF)
Destination Clock: clk rising
```

Data Path: A to out_3

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	3	1.218	0.706	A_IBUF (A_IBUF)
LUT4_L:I0->LO	1	0.704	0.179	Result<3>_SW0 (N01)
LUT2:I1->O	1	0.704	0.000	Result<3> (Result<3>)
FDCE:D		0.308		out_3

Total		3.819ns (2.934ns logic, 0.885ns route) (76.8% logic, 23.2% route)		

```
=====
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'
Total number of paths / destination ports: 4 / 4
=====
```

```
Offset:          4.496ns (Levels of Logic = 1)
Source:          out_0 (FF)
Destination:     out<0> (PAD)
Source Clock:    clk rising
```

Data Path: out_0 to out<0>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDCE:C->Q	5	0.591	0.633	out_0 (out_0)
OBUF:I->O		3.272		out_0_OBUF (out<0>)

Total		4.496ns (3.863ns logic, 0.633ns route) (85.9% logic, 14.1% route)		

```
=====
Total REAL time to Xst completion: 20.00 secs
Total CPU time to Xst completion: 20.72 secs
=====
```

-->

Up/down counter with enable/reset Gate level Time summary

Timing Summary:

Speed Grade: -4

Minimum period: 3.123ns (Maximum Frequency: 320.205MHz)
Minimum input arrival time before clock: 3.819ns
Maximum output required time after clock: 4.496ns
Maximum combinational path delay: No path found

Timing Detail:

All values displayed in nanoseconds (ns)

Timing constraint: Default period analysis for Clock 'clk'

Clock period: 3.116ns (frequency: 320.924MHz)

Total number of paths / destination ports: 19 / 4

Delay: 3.116ns (Levels of Logic = 2)

Source: state_0 (FF)

Destination: state_1 (FF)

Source Clock: clk rising

Destination Clock: clk rising

Data Path: state_0 to state_1

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDCE:C->Q	15	0.591	1.192	state_0 (state_0)
LUT2:I0->O	1	0.704	0.000	Mmux_state_mux00004_G (N12)
MUXF5:I1->O	1	0.321	0.000	Mmux_state_mux00004 (state_mux0000<0>)
FDCE:D		0.308		state_3
Total		3.116ns (1.924ns logic, 1.192ns route) (61.7% logic, 38.3% route)		

Timing constraint: Default OFFSET IN BEFORE for Clock 'clk'

Total number of paths / destination ports: 10 / 7

Offset: 3.299ns (Levels of Logic = 3)

Source: A (PAD)

Destination: state_1 (FF)

Destination Clock: clk rising

Data Path: A to state_1

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	6	1.218	0.748	A_IBUF (A_IBUF)
LUT2:I1->O	1	0.704	0.000	Mmux_state_mux00004_G (N12)
MUXF5:I1->O	1	0.321	0.000	Mmux_state_mux00004 (state_mux0000<0>)
FDCE:D		0.308		state_3
Total		3.299ns (2.551ns logic, 0.748ns route) (77.3% logic, 22.7% route)		

Up/down counter with enable/reset State based behavioral code's Time summary

=====
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'

Total number of paths / destination ports: 32 / 11
=====

Offset: 6.145ns (Levels of Logic = 2)

Source: state_3 (FF)

Destination: seg<2> (PAD)

Source Clock: clk rising

Data Path: state_3 to seg<2>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDCE:C->Q	13	0.591	1.158	state_3 (state_3)
LUT4:I0->O	1	0.704	0.420	BC1/Mrom_segments21 (seg_2_OBUF)
OBUF:I->O		3.272		seg_2_OBUF (seg<2>)

Total		6.145ns (4.567ns logic, 1.578ns route) (74.3% logic, 25.7% route)		

=====

Total REAL time to Xst completion: 22.00 secs

Total CPU time to Xst completion: 21.43 secs

Up/down counter with enable/reset State based behavioral code's Time
summary

Timing Summary:

Speed Grade: -4

Minimum period: 3.033ns (Maximum Frequency: 329.707MHz)
Minimum input arrival time before clock: 3.825ns
Maximum output required time after clock: 4.450ns
Maximum combinational path delay: No path found

Timing Detail:

All values displayed in nanoseconds (ns)

Timing constraint: Default period analysis for Clock 'clk'

Clock period: 3.033ns (frequency: 329.707MHz)

Total number of paths / destination ports: 10 / 4

Delay: 3.033ns (Levels of Logic = 2)

Source: out_1 (FF)

Destination: out_0 (FF)

Source Clock: clk rising

Destination Clock: clk rising

Data Path: out_1 to out_0

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDC:C->Q	4	0.591	0.622	out_1 (out_1)
LUT3_L:I2->LO	1	0.704	0.104	Mmux_out_mux00002_SW0 (N4)
LUT4:I3->O	1	0.704	0.000	Mmux_out_mux00002 (out_mux0000<0>)
FDC:D		0.308		out_0
Total		3.033ns (2.307ns logic, 0.726ns route) (76.1% logic, 23.9% route)		

n-bit counter with enable/reset behavioral code's Time summary

=====
Timing constraint: Default OFFSET IN BEFORE for Clock 'clk'

Total number of paths / destination ports: 16 / 4

Offset: 3.825ns (Levels of Logic = 3)

Source: select<0> (PAD)

Destination: out_3 (FF)

Destination Clock: clk rising

Data Path: select<0> to out_3

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	7	1.218	0.787	select_0_IBUF (select_0_IBUF)
LUT3_L:I1->LO	1	0.704	0.104	Mmux_out_mux00004_SW0 (N2)
LUT4:I3->O	1	0.704	0.000	Mmux_out_mux00004 (out_mux0000<3>)
FDC:D		0.308		out_3
Total		3.825ns	(2.934ns logic, 0.891ns route)	(76.7% logic, 23.3% route)

=====

=====
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'

Total number of paths / destination ports: 4 / 4

Offset: 4.450ns (Levels of Logic = 1)

Source: out_2 (FF)

Destination: out<2> (PAD)

Source Clock: clk rising

Data Path: out_2 to out<2>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDC:C->Q	4	0.591	0.587	out_2 (out_2)
OBUF:I->O		3.272		out_2_OBUF (out<2>)
Total		4.450ns	(3.863ns logic, 0.587ns route)	(86.8% logic, 13.2% route)

=====

Total REAL time to Xst completion: 20.00 secs

Total CPU time to Xst completion: 20.66 secs

n-bit counter with enable/reset behavioral code's Time summary

Critical analysis/conclusion

In this lab I learned a lot about how to build counters, firstly I created the up counter in prelab task with state diagram and state table and its equation, then I used all the control inputs to create a counter which counts up as well as down and also has reset and enable key to reset or pause it respectively. I also coded it in state-based behavioral coding and found its result similar to gate level. In the post lab, lastly I created an n-bit counter, I lastly checked its resource utilization, time summary and analyzed it. The overall lab helped a lot and cleared the ambiguities about counters.

Lab 12

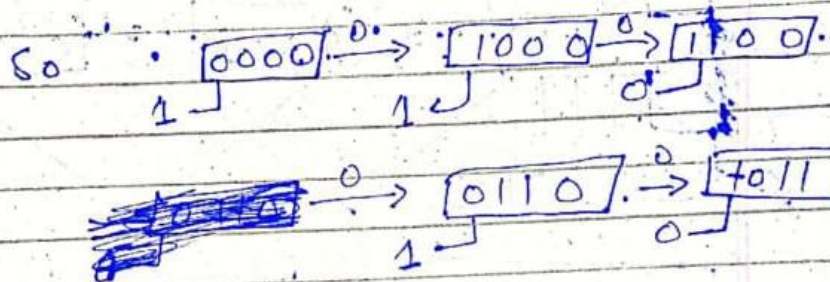
Pre-Lab task:

Types of ~~shift~~ Registers:-

Serial in Serial out:-

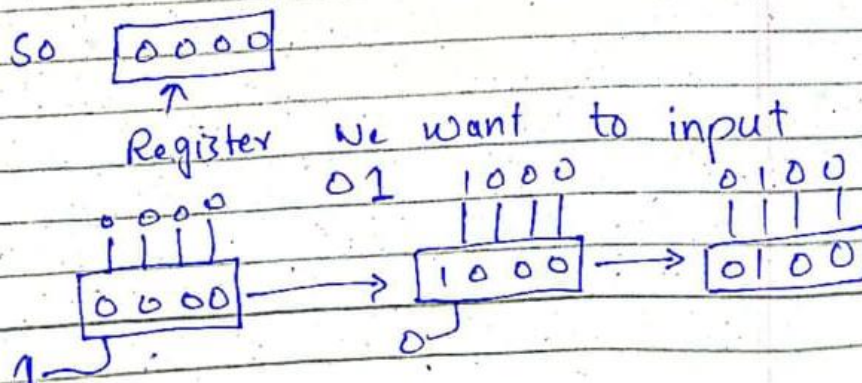
Data is entered serially one by one at the Mshin or Lshin and outputted serially.

0000 we want to input 1011
↑
Register



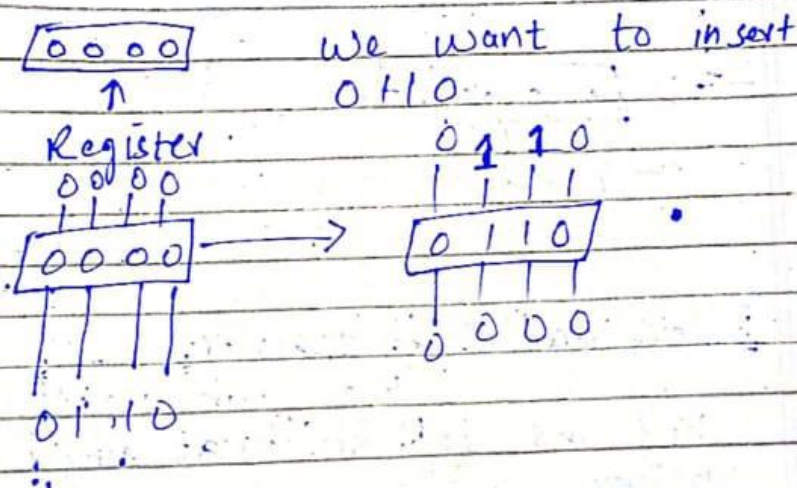
Serial in parallel out:-

Data is inputted serially one by one and is outputted all at same time as parallel.



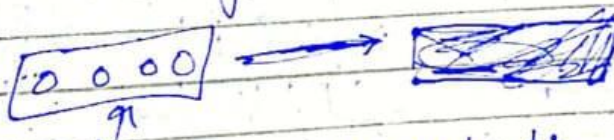
Parallel in Parallel out:-

parallel and is also outputted parallel.
Data is entered parallel.

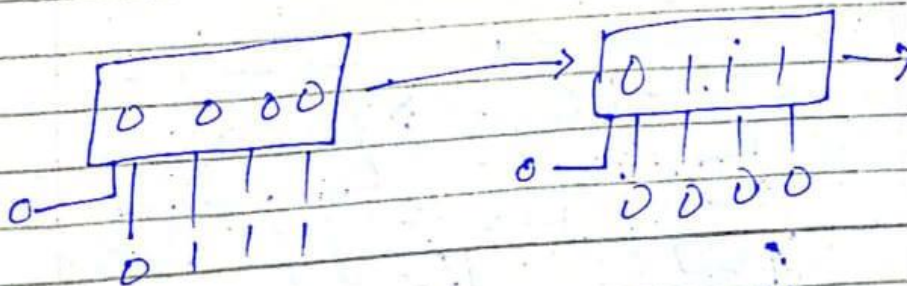


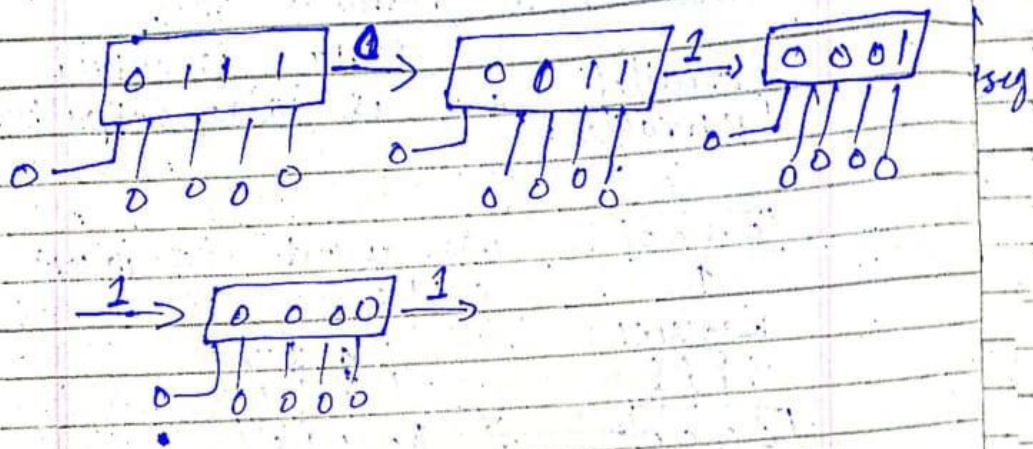
Parallel in serial out:-

Data is inputted parallel but outputted serially.



Register we want to insert 0111

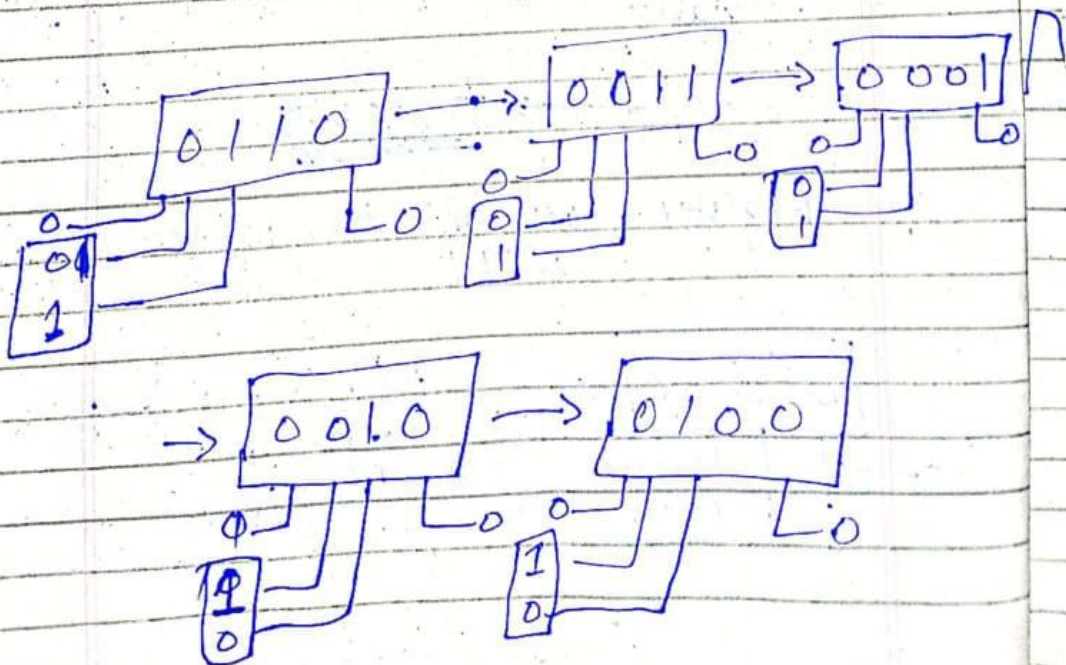




Bidirectional shift register:-

We can right shift and left shift that single register.

01 Right shift
10 Left shift

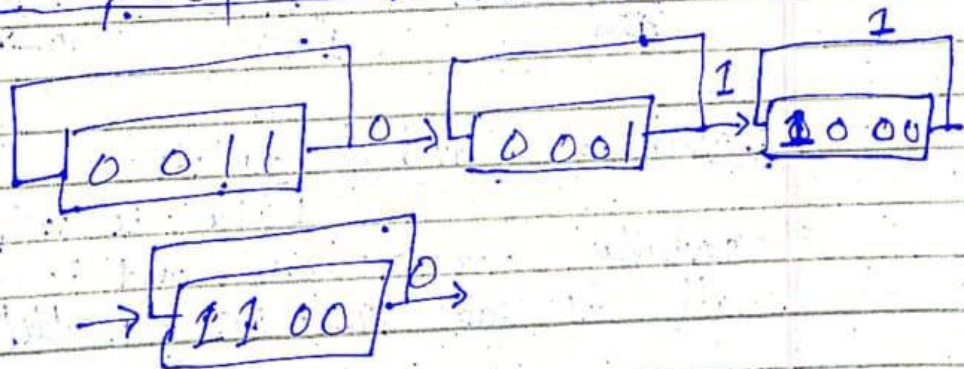
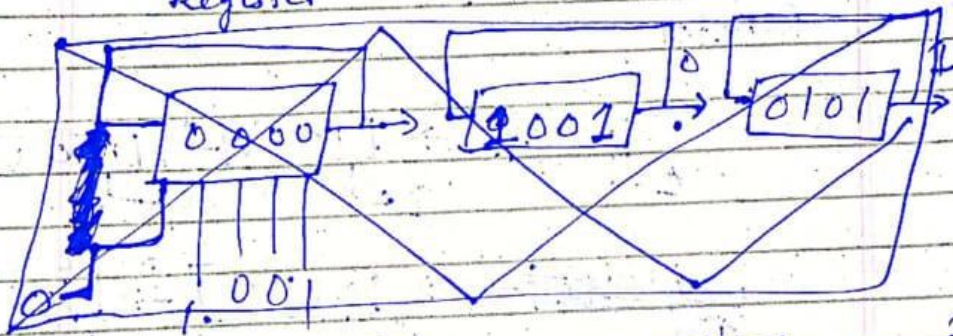


Rotate Shift Register:-

The output of serial out register is put back in the input of register.

0000 we input 1001

Register

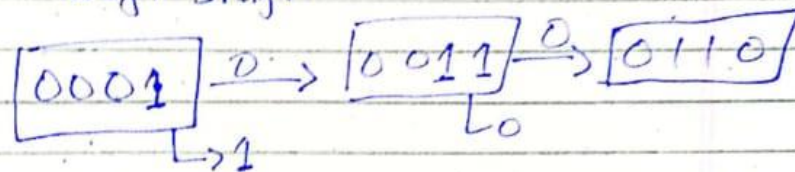


Multiple shifting:-

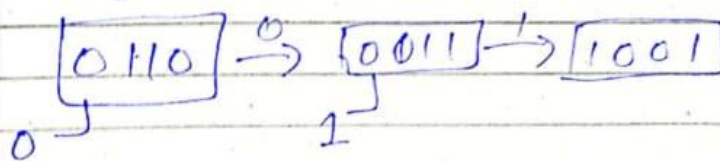
We have total of three type of shifting one is Right shifting, another Left shift and lastly parallel shift or parallelly output.

Examples.

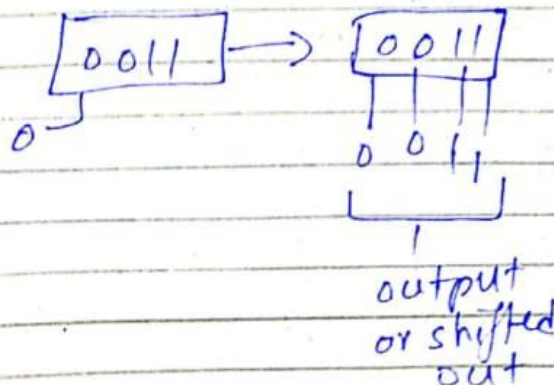
Left shift

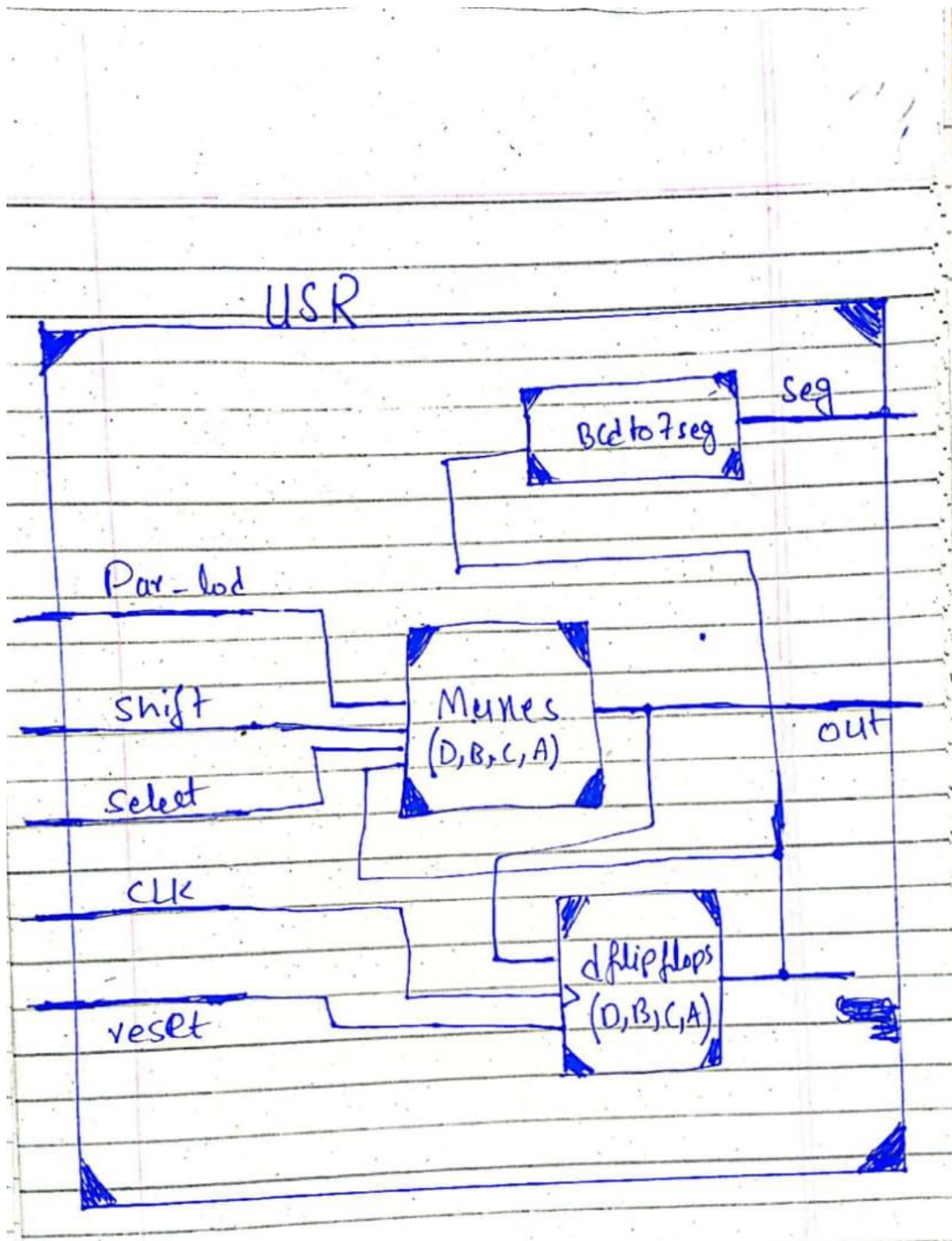


Right shift:-



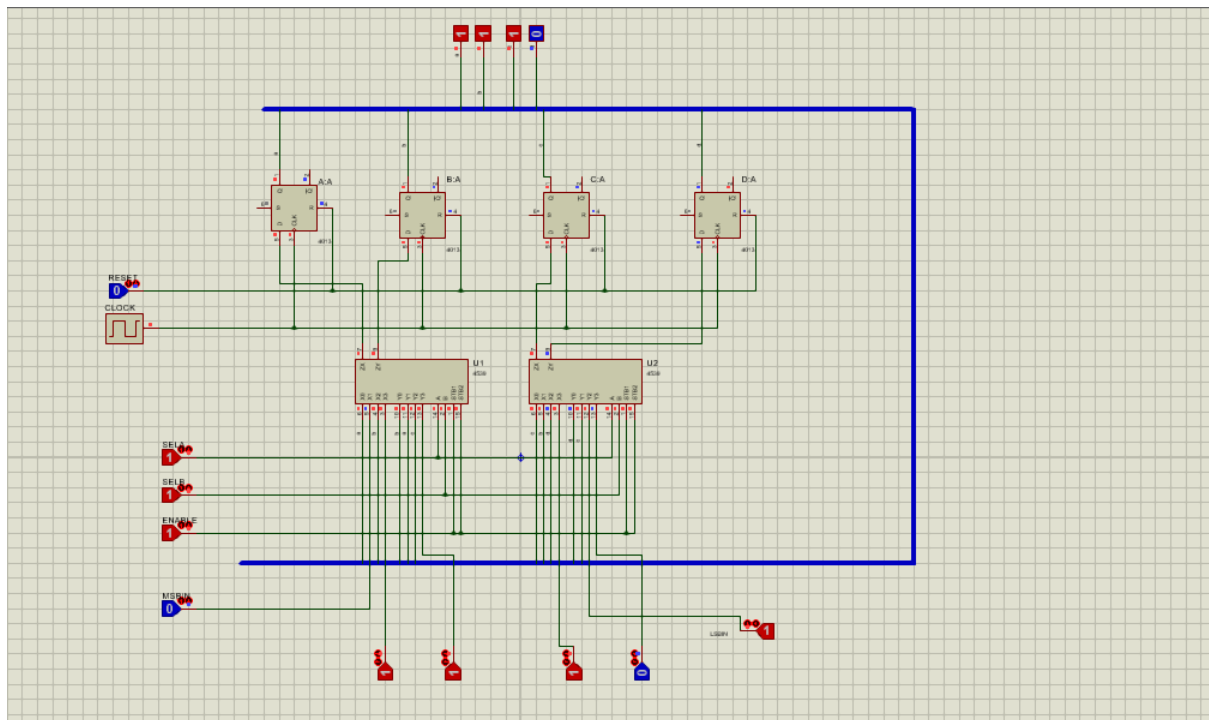
parallel output/shift



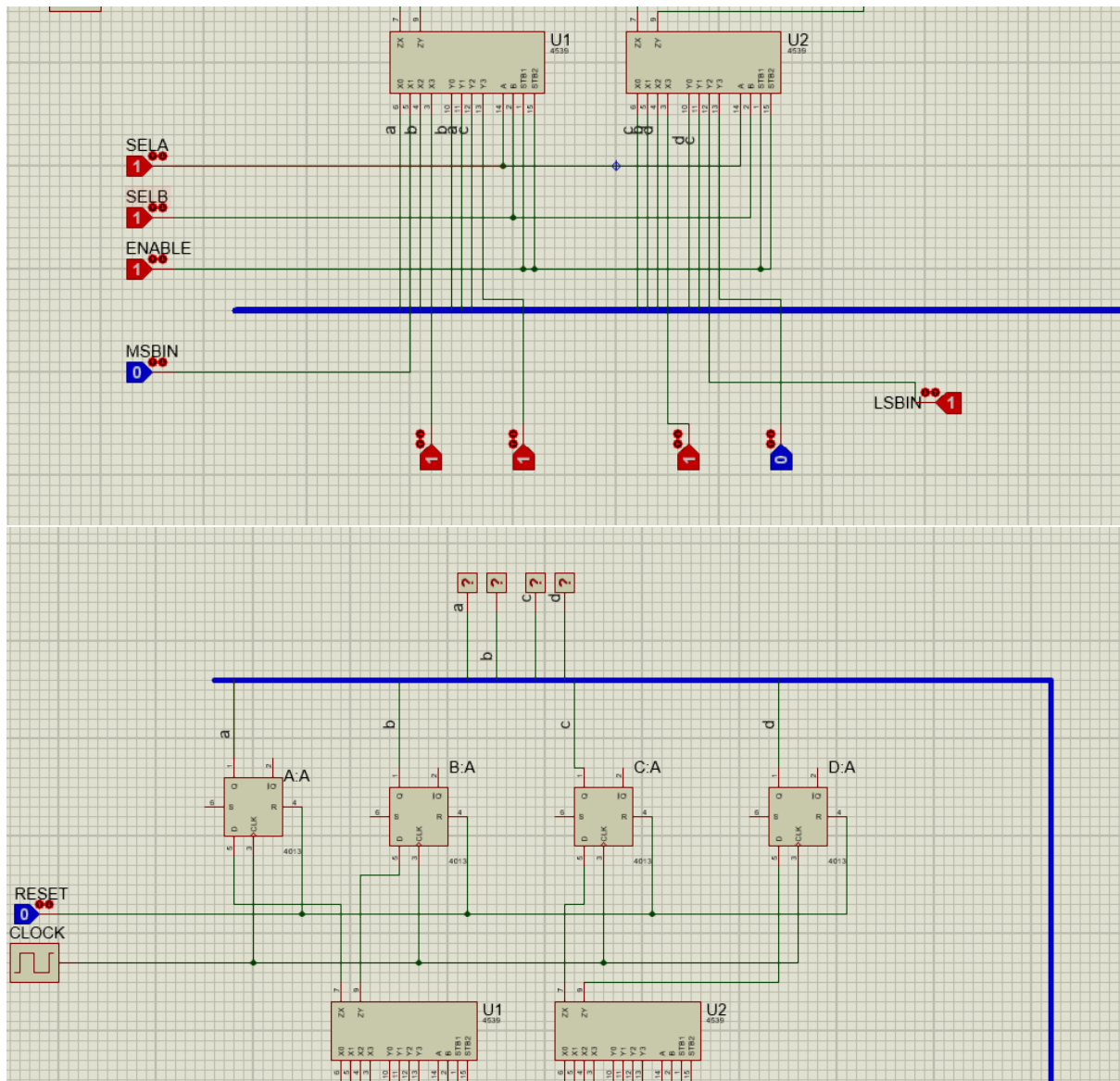


Block Diagram of Universal Shift Register

In-lab task 1:



Circuit Diagram of Universal shift Register



Circuit Diagram of Universal shift Register

```

20 //////////////////////////////////////////////////
21 module registerrrrr( par_load,select,shift,clk,reset,out,seg0,seg1,seg2,seg3);
22 input clk,reset;
23 assign anode=4'b1111;//for SEVENsegmentDisp on FPGA
24 input [1:0]select,shift;//SelectA and SelectB, MSBin for rightshift,LSBin for leftshift
25 input [3:0]par_load;//parrallel load
26 output [3:0]out;//output
27 //every register will requires 4 seven segment display to diplay register values in 1's and 0's
28 output [7:0]seg0,seg1,seg2,seg3;//one for each bit of 4bit shift register
29
30 wire [3:0]temp;//for output of flipflops
31 wire [3:0]temp2;//for output of multiplexers
32
33 wire [3:0]multiD;//for multiplexer of Dflipflop
34 wire [3:0]multiC;//for multiplexer of Cflipflop
35 wire [3:0]multiB;//for multiplexer of Bflipflop
36 wire [3:0]multiA;//for multiplexer of Aflipflop
37
38
39 assign multiD={par_load[0],shift[1],temp[1],temp[0]};//option for MuxD
40 multiplexerrrr MD(multiD,select,temp2[0]);//mux for Dflipflop
41
42 assign multiC={par_load[1],temp[0],temp[2],temp[1]};//option for MuxC
43 multiplexerrrr MC(multiC,select,temp2[1]);//mux for Cflipflop
44
45
46 assign multiB={par_load[2],temp[1],temp[3],temp[2]};//option for MuxB
47 multiplexerrrr MB(multiB,select,temp2[2]);//mux for Bflipflop
48
49 assign multiA={par_load[3],temp[2],shift[0],temp[3]};//option for MuxA
50 multiplexerrrr MA(multiA,select,temp2[3]);//mux for Aflipflop
51
52 dflipflop flipD(temp[0],temp2[0],clk,reset);//flipflopD
53 dflipflop flipC(temp[1],temp2[1],clk,reset);//flipflopC
54 dflipflop flipB(temp[2],temp2[2],clk,reset);//flipflopB
55 dflipflop flipA(temp[3],temp2[3],clk,reset);//flipflopA
56
57 assign out[3:0]= temp[3:0];
58 bcdto7segment BCD0({1'b0,1'b0,1'b0,out[0]},seg0);//output of flipflopD in seven segment
59 bcdto7segment BCD1({1'b0,1'b0,1'b0,out[1]},seg1);//output of flipflopC in seven segment
60 bcdto7segment BCD2({1'b0,1'b0,1'b0,out[2]},seg2);//output of flipflopB in seven segment
61 bcdto7segment BCD3({1'b0,1'b0,1'b0,out[3]},seg3);//output of flipflopA in seven segment
62 endmodule

```

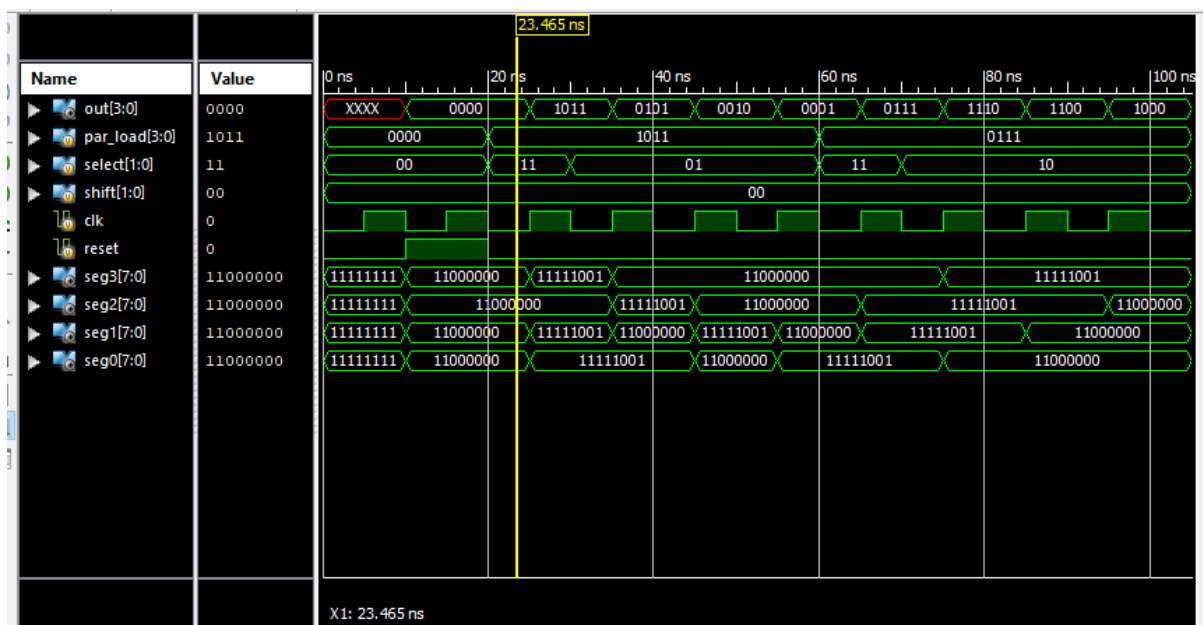
Structural code of Universal shift Register

```

25 module registertestbench;
26
27     // Inputs
28     reg [3:0] par_load;
29     reg [1:0] select;
30     reg [1:0] shift;
31     reg clk;
32     reg reset;
33
34     // Outputs
35     wire [3:0] out;
36     wire [7:0] seg0,seg1,seg2,seg3;
37     // Instantiate the Unit Under Test (UUT)
38     registerrrrr uut (
39         .par_load(par_load),
40         .select(select),
41         .shift(shift),
42         .clk(clk),
43         .reset(reset),
44         .out(out),
45         .seg0(seg0),
46         .seg1(seg1),
47         .seg2(seg2),
48         .seg3(seg3)
49     );
50
51     initial begin
52         // Initialize Inputs
53         par_load = 0;//parrallel load 4'b0000;
54         select = 0;    //select 2'b00;
55         shift = 0;    //shift 2'b00;
56         clk = 0;
57         reset = 0;          #10;
58         reset =1;          #10;//reset
59         reset=0;
60         select=2'b11;      //parrallel load selection
61         par_load=4'b1011;  #10;//parrallel load
62         select=2'b01;      #30//rightshift selection
63         select=2'b11;      //parrallel load selection
64         par_load=4'b0111;  #10;//parrallel load
65         select=2'b10;      #35;//leftshift slection
66         $finish;
67
68     end
69
70     always #5 clk=~clk;
71 endmodule
72
73

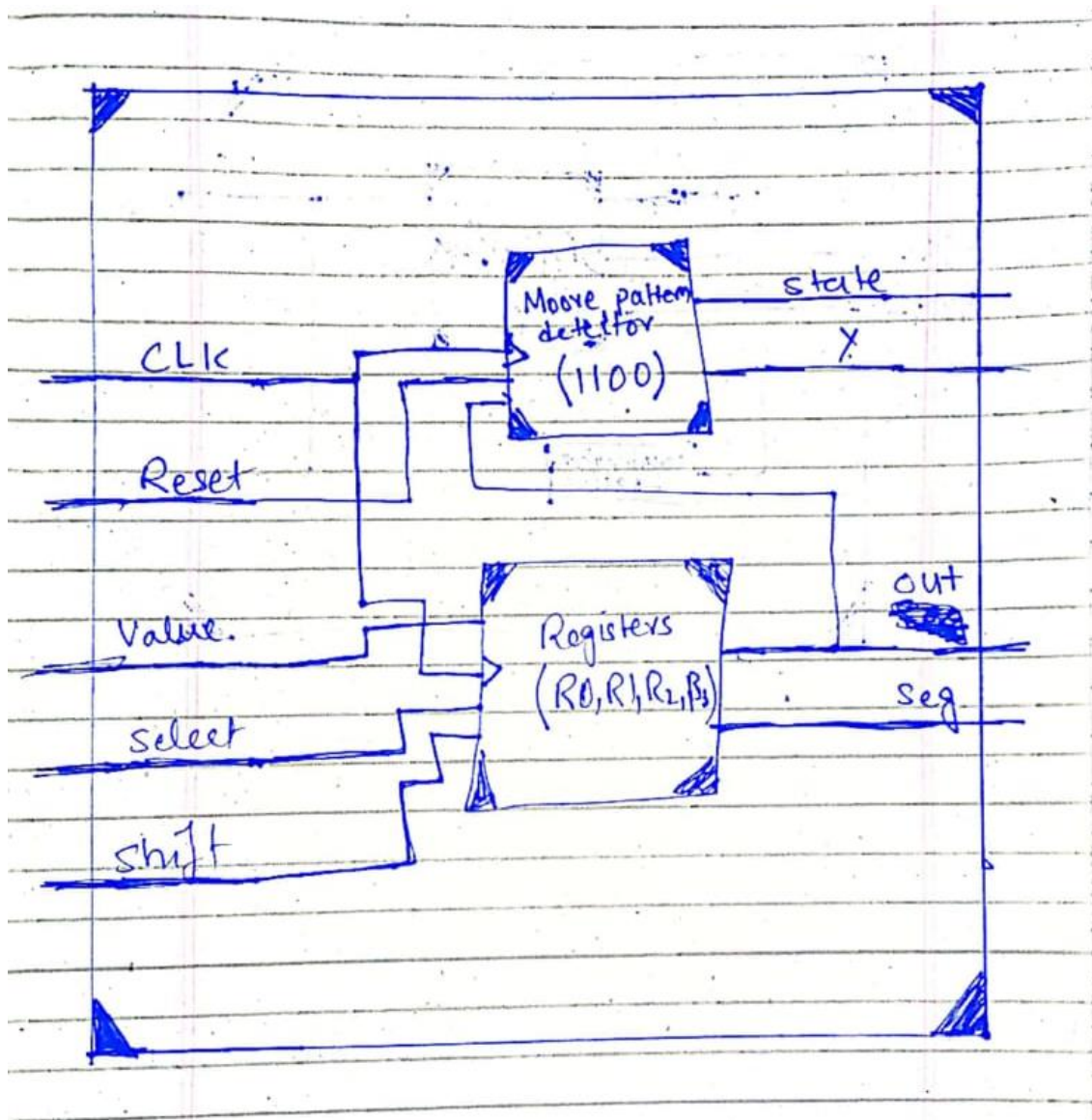
```

Testbench for Universal shift Register



Structural level Universal shift register Graph

In lab task2:



Block diagram of Universal shift register Graph

```

21 module tsk2(
22     value,reset,clk,currentstate,y );
23 parameter A=3'b000,B=3'b001,C=3'b010,D=3'b011,E=3'b100;//states
24
25 input value,reset,clk;//inputs
26 output reg y;//output
27 output reg [2:0]currentstate;//nextstate or output state
28
29 always@(posedge clk or posedge reset )//positive edge clock and reset
30 begin
31 if(reset==1)begin currentstate<=A; end//if reset==1 curentstate becomes A
32 else begin
33 case (currentstate)
34 //coresponding state's next state based on input
35 A: if( value==0 )begin currentstate<=A; end else begin currentstate<=B; end
36 B: if( value==0 )begin currentstate<=A; end else begin currentstate<=C; end
37 C: if( value==0 )begin currentstate<=D; end else begin currentstate<=C; end
38 D: if( value==0 )begin currentstate<=E; end else begin currentstate<=B; end
39 E: if( value==0 )begin currentstate<=A; end else begin currentstate<=B; end
40 default: begin currentstate<=2'b00; end
41 endcase
42 end
43 end
44 end
45
46 //output depends only on state
47 //it is declared seperately as declaring in the above blocks creates
48 //a bug
49 always@(currentstate)
50 begin
51 case(currentstate)
52 A: y <=0;
53 B: y <= 0;
54 C: y <= 0;
55 D: y <= 0;
56 E: y <= 1;//the state where output is 1
57 default: y= 0;
58 endcase
59 end
60 endmodule
61

```

Moore machine code

```

20 //////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
21 module shifter(value,shift,select,clk,reset,state,y
22 ,out0,out1,out2,out3,seg0,seg1,seg2,seg3 );
23 input [15:0] value;//input sequence
24 input [1:0]shift,select;//SelectA and SelectB, MSBin for rightshift,LSBin for leftshift for registers
25 input clk,reset;//clock and reset
26 output [3:0]out0,out1,out2,out3;//output of each register
27 output [2:0]state;//current state
28 output y;//output pattern detected
29 //we need 16 seven segment displays for this
30 //Note: one register values can be shown in one seven segment display but if the bits represent number greater
31 //than 9 then it is not possible, thats why each bit as 1's and 0's is is ouputted onto th 7 seg displays
32 output [31:0] seg0,seg1,seg2,seg3;//for the display of each register 0's and 1's on sevesegment display
33 //registers with its values as output and also in SEVEN segment codes
34 //the values from R3 that getsout at the right gets in into the R2 and so on, the value of R0 that gets
35 //out is enter to module for pattern detector.
36 register R3(value[15:12],select,shift,clk,reset,out3,seg3[7:0],seg3[15:8],seg3[23:16],seg3[31:24]);
37 register R2(value[11:8],select,{1'b0,out3[0]},clk,reset,out2,seg2[7:0],seg2[15:8],seg2[23:16],seg2[31:24]);
38 register R1(value[7:4],select,{1'b0,out2[0]},clk,reset,out1,seg1[7:0],seg1[15:8],seg1[23:16],seg1[31:24]);
39 register R0(value[3:0],select,{1'b0,out1[0]},clk,reset,out0,seg0[7:0],seg0[15:8],seg0[23:16],seg0[31:24]);
40
41 tsk2 M1(out0[0],reset,clk,state,y);// the pattern detectot
42 endmodule
43

```

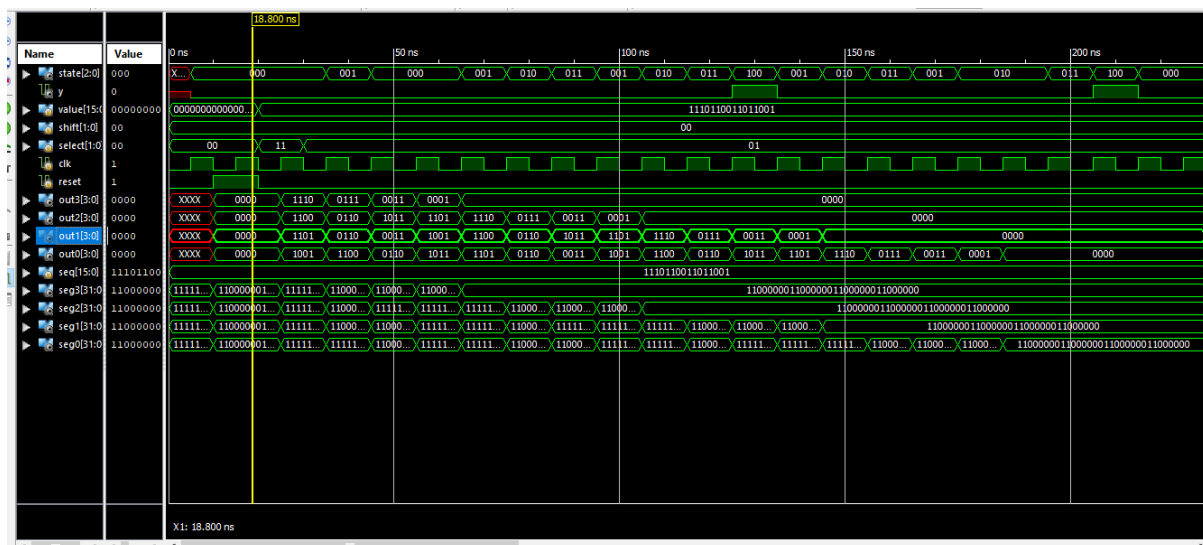
Shifting values to Moore machine via Universal shift register code

```

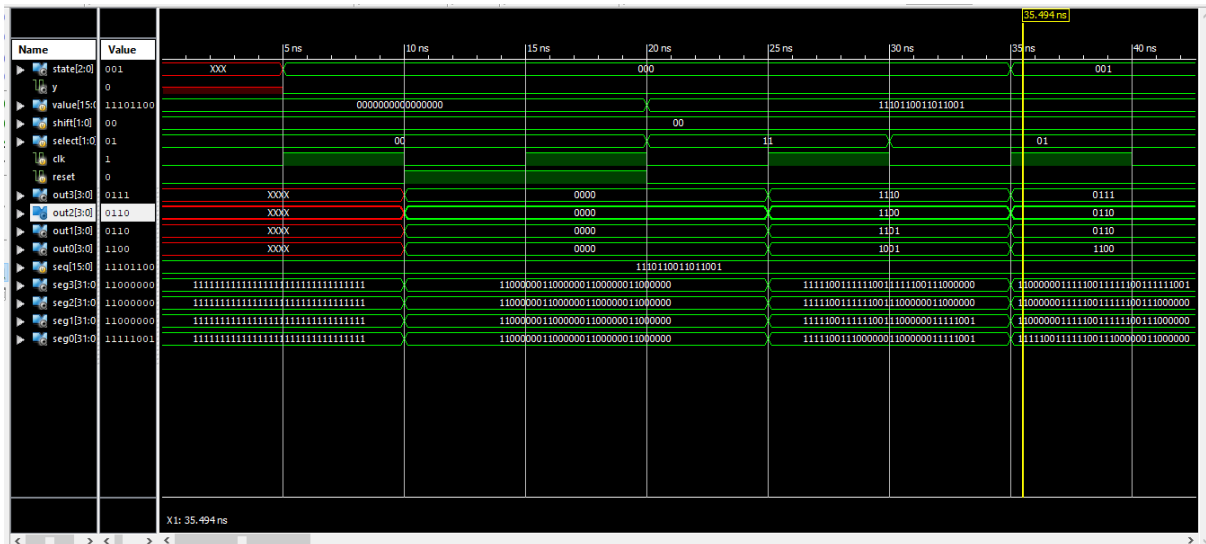
25 module testbnchforshifter;
26     // Inputs
27     reg [15:0] value;
28     reg [1:0] shift;
29     reg [1:0] select;
30     reg clk;
31     reg reset;
32
33     // Outputs
34     wire [2:0] state;
35     wire y;
36     wire [3:0] out0;
37     wire [3:0] out1;
38     wire [3:0] out2;
39     wire [3:0] out3;
40     wire [31:0] seg0, seg1, seg2, seg3;
41
42     // Instantiate the Unit Under Test (UUT)
43     shifter uut (
44         .value(value),
45         .shift(shift),
46         .select(select),
47         .clk(clk),
48         .reset(reset),
49         .state(state),
50         .y(y),
51         .out0(out0),
52         .out1(out1),
53         .out2(out2),
54         .out3(out3),
55         .seg0(seg0),
56         .seg1(seg1),
57         .seg2(seg2),
58         .seg3(seg3)
59     );
60     reg [15:0] seq = 16'b1110110011011001;
61     initial begin
62         // Initialize Inputs
63         value = 0; //input sequenc 16 bits zeros
64         shift = 0; // selection no change
65         select = 0; //MSBin, LSBin, both zero
66         clk = 0;
67         reset = 0;      #10;
68         reset = 1;      #10; //reset
69         reset = 0;
70         select=2'b11; //selection mode parrallel load
71         value=seq;    #10; // sequence loaded
72         select=2'b01; #200; // selection mode rightshifting
73         $finish;
74     end
75     always #5 clk=~clk;
76 endmodule
77
78

```

Shifting values to Moore machine via Universal shift register code's



Shifting values to Moore machine via Universal shift register code's Graph 1



Shifting values to Moore machine via Universal shift register code's Graph 2

Post-lab task 1:

```

21 module nbitshifter(value,select,shift,clk,reset,out
22 );
23 parameter bits=4;//no of bits
24
25 input [bits-1:0] value;//parrallel load
26 input [1:0]select,shift;////SelectA and SelectB, MSBin for rightshift,LSBin for leftshift for register
27 input clk,reset;//clock and reset
28 output reg [bits-1:0] out;//output
29
30 //positive edge clock and reset
31 always @(posedge clk , posedge reset )
32 begin
33 if (reset)begin out<=0; end//reseting
34 else begin
35 case (select)//selction
36 2'b00: begin out<=out; end //nochange
37 2'b01: begin out<=out >> 1; end//righthshift
38 2'b10: begin out<=out << 1; end//leftshift
39 2'b11: begin out<=value; end//parrallel load
40 default: out<=out;//no change
41 endcase
42 end
43 end
44 endmodule
45

```

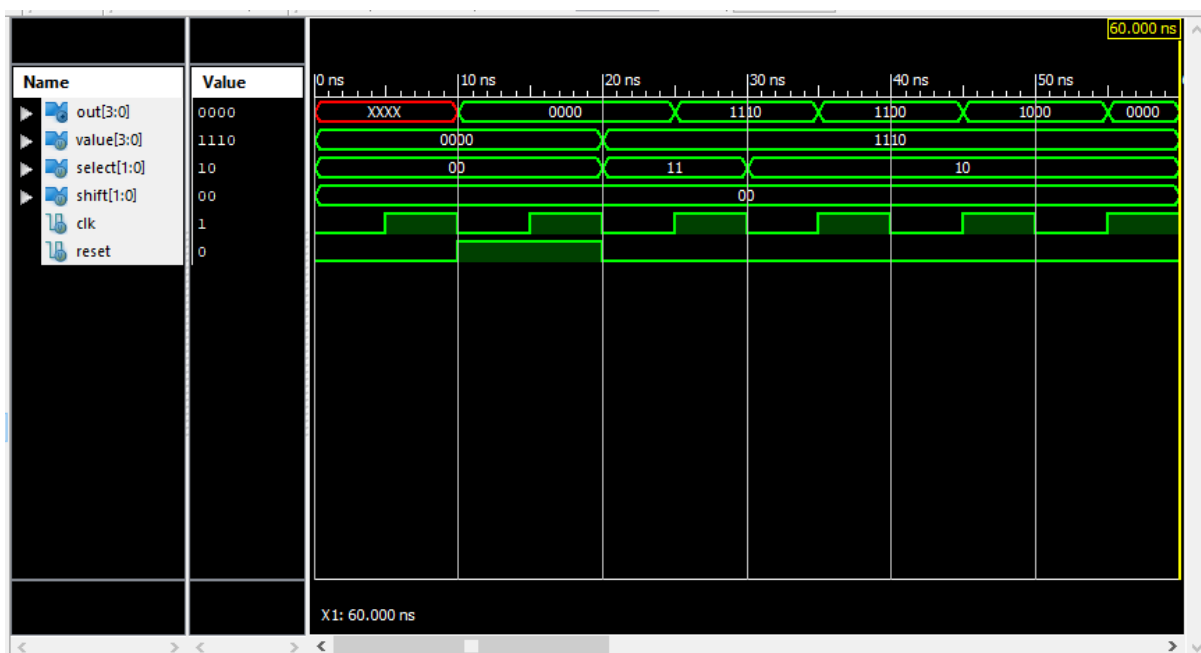
n-bit universal shift register behavioral code

```

24
25 module nbittestbench;|
26     // Inputs
27     reg [3:0] value;
28     reg [1:0] select;
29     reg [1:0] shift;
30     reg clk;
31     reg reset;
32
33     // Outputs
34     wire [3:0] out;
35
36     // Instantiate the Unit Under Test (UUT)
37     nbitchifter uut (
38         .value(value),
39         .select(select),
40         .shift(shift),
41         .clk(clk),
42         .reset(reset),
43         .out(out)
44     );
45
46     initial begin
47         // Initialize Inputs
48         value = 0; //parallel load
49         select = 0; // selection no change
50         shift = 0; //MSBin, LSBin, both zero
51         clk = 0;
52         reset = 0; #10;
53         reset = 1; #10; //reset
54         reset = 0;
55         select = 2'b11; //selection mode parallel load
56         value = 4'b1110; #10; //parallel load
57         select = 2'b10; #30; //left shifting
58         $finish;
59
60     end
61     always #5 clk = ~clk;
62 endmodule

```

n-bit universal shift register behavioral code's Testbench



n-bit universal shift register behavioral code's graph

Post-lab task 2:

Device utilization summary:

Selected Device : 3s500efg320-4

Number of Slices:	4	out of	4656	0%
Number of Slice Flip Flops:	4	out of	9312	0%
Number of 4 input LUTs:	8	out of	9312	0%
Number of IOs:	46			
Number of bonded IOBs:	46	out of	232	19%
Number of GCLKs:	1	out of	24	4%

Structural level Universal shift register resource utilization

Device utilization summary:

Selected Device : 3s500efg320-4

Number of Slices:	4	out of	4656	0%
Number of Slice Flip Flops:	4	out of	9312	0%
Number of 4 input LUTs:	8	out of	9312	0%
Number of IOs:	14			
Number of bonded IOBs:	12	out of	232	5%
Number of GCLKs:	1	out of	24	4%

n-bit Universal shift register behavioral code's resource utilization

Timing Summary:

Speed Grade: -4

Minimum period: 2.760ns (Maximum Frequency: 362.319MHz)
Minimum input arrival time before clock: 3.483ns
Maximum output required time after clock: 4.620ns
Maximum combinational path delay: No path found

Timing Detail:

All values displayed in nanoseconds (ns)

Timing constraint: Default period analysis for Clock 'clk'

Clock period: 2.760ns (frequency: 362.319MHz)

Total number of paths / destination ports: 10 / 4

Delay: 2.760ns (Levels of Logic = 2)

Source: flipC/Q (FF)

Destination: flipC/Q (FF)

Source Clock: clk rising

Destination Clock: clk rising

Data Path: flipC/Q to flipC/Q

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDC:C->Q	8	0.591	0.836	flipC/Q (flipC/Q)
LUT3:I1->O	1	0.704	0.000	MC/Mmux_out_2_f5_F (N6)
MUXF5:I0->O	1	0.321	0.000	MC/Mmux_out_2_f5 (temp2<1>)
FDC:D		0.308		flipC/Q
Total		2.760ns (1.924ns logic, 0.836ns route) (69.7% logic, 30.3% route)		

Timing constraint: Default OFFSET IN BEFORE for Clock 'clk'

Total number of paths / destination ports: 18 / 4

Offset: 3.483ns (Levels of Logic = 3)

Source: select<0> (PAD)

Destination: flipB/Q (FF)

Destination Clock: clk rising

Data Path: select<0> to flipB/Q

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	8	1.218	0.932	select_0_IBUF (select_0_IBUF)
LUT3:I0->O	1	0.704	0.000	MD/Mmux_out_3 (MD/Mmux_out_3)
MUXF5:I1->O	1	0.321	0.000	MD/Mmux_out_2_f5 (temp2<0>)
FDC:D		0.308		flipD/Q
Total		3.483ns (2.551ns logic, 0.932ns route) (73.2% logic, 26.8% route)		

Structural level Universal shift register Time summary

```
=====
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'
Total number of paths / destination ports: 20 / 20
=====
```

```
Offset:          4.620ns (Levels of Logic = 1)
Source:          flipC/Q (FF)
Destination:     seg1<5> (PAD)
Source Clock:    clk rising
```

Data Path: flipC/Q to seg1<5>

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDC:C->Q	8	0.591	0.757	flipC/Q (flipC/Q)
OBUF:I->O		3.272		seg1_5_OBUF (seg1<5>)

Total		4.620ns (3.863ns logic, 0.757ns route) (83.6% logic, 16.4% route)		

```
=====
Total REAL time to Xst completion: 23.00 secs
Total CPU time to Xst completion: 23.67 secs
```

-->

Structural level Universal shift register Time summary
--

Timing Summary:

Speed Grade: -4

Minimum period: 3.033ns (Maximum Frequency: 329.707MHz)
Minimum input arrival time before clock: 3.825ns
Maximum output required time after clock: 4.450ns
Maximum combinational path delay: No path found

Timing Detail:

All values displayed in nanoseconds (ns)

Timing constraint: Default period analysis for Clock 'clk'

Clock period: 3.033ns (frequency: 329.707MHz)

Total number of paths / destination ports: 10 / 4

Delay: 3.033ns (Levels of Logic = 2)

Source: out_1 (FF)

Destination: out_0 (FF)

Source Clock: clk rising

Destination Clock: clk rising

Data Path: out_1 to out_0

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
FDC:C->Q	4	0.591	0.622	out_1 (out_1)
LUT3_L:I2->LO	1	0.704	0.104	Mmux_out_mux00002_SW0 (N4)
LUT4:I3->O	1	0.704	0.000	Mmux_out_mux00002 (out_mux0000<0>)
FDC:D		0.308		out_0

Total 3.033ns (2.307ns logic, 0.726ns route)
(76.1% logic, 23.9% route)

Timing constraint: Default OFFSET IN BEFORE for Clock 'clk'

Total number of paths / destination ports: 16 / 4

Offset: 3.825ns (Levels of Logic = 3)

Source: select<0> (PAD)

Destination: out_3 (FF)

Destination Clock: clk rising

Data Path: select<0> to out_3

Cell:in->out	fanout	Gate Delay	Net Delay	Logical Name (Net Name)
IBUF:I->O	7	1.218	0.787	select_0_IBUF (select_0_IBUF)
LUT3_L:I1->LO	1	0.704	0.104	Mmux_out_mux00004_SW0 (N2)
LUT4:I3->O	1	0.704	0.000	Mmux_out_mux00004 (out_mux0000<3>)
FDC:D		0.308		out_3

Total 3.825ns (2.934ns logic, 0.891ns route)
(76.7% logic, 23.3% route)

n-bit Universal shift register behavioral code's Time summary

```

=====
Timing constraint: Default OFFSET OUT AFTER for Clock 'clk'
Total number of paths / destination ports: 4 / 4
=====
Offset:                4.450ns (Levels of Logic = 1)
Source:                out_2 (FF)
Destination:          out<2> (PAD)
Source Clock:          clk rising

Data Path: out_2 to out<2>

Cell:in->out      fanout      Gate      Net
                        Delay      Delay      Logical Name (Net Name)
-----
FDC:C->Q           4      0.591      0.587      out_2 (out_2)
OBUF:I->O           3.272                        out_2_OBUF (out<2>)
-----
Total                                4.450ns (3.863ns logic, 0.587ns route)
                                           (86.8% logic, 13.2% route)
=====

Total REAL time to Xst completion: 21.00 secs
Total CPU time to Xst completion: 20.73 secs

```

n-bit Universal shift register behavioral code's Time summary

Critical analysis/conclusion

In this lab I firstly found about different types of registers and its shifting. Then I made the universal shift register and its block diagram. The universal shift register had selection, MSBin, LSBin, parallel load, reset, clock inputs and its values as bits output and also in 7 seven segment code. the register was able to shift both right and left also in parallel out. Also we used the register as shifting device to the Moore machine, and drew its block diagram. In post lab I created an n bit universal shift register, with the same functionality as gate level USR but in behavioral code. I lastly checked its resource utilization, time summary and analyzed it. The lab was very helpful in clearing the concepts of register and its usage.